arXiv:2509.23459v2 [cs.CR] 30 Sep 2025

MaskSQL: Safeguarding Privacy for LLM-Based Text-to-SQL via Abstraction

Sepideh Abedini[1,2] **Shubhankar Mohapatra**[1] **D. B. Emerson**[2]
Masoumeh Shafieinejad[2] **Jesse C. Cresswell**[3] **Xi He**[1,2]
University of Waterloo[1]
{sepideh.abedini,shubhankar.mohapatra,xi.he}@uwaterloo.ca
Vector Institute[2] Layer 6 AI[3]
{david.emerson, masoumeh}@vectorinstitute.ai jesse@layer6.ai

Abstract

Large language models (LLMs) have shown promising performance on tasks that require reasoning, such as text-to-SQL translation, code generation, and debugging. However, regulatory frameworks with strict privacy requirements constrain their integration into sensitive systems. State-of-the-art LLMs are also proprietary, costly, and resource-intensive, making local deployment impractical. Consequently, utilizing such LLMs often requires sharing data with third-party providers, raising privacy concerns and risking noncompliance with regulations. Although fine-tuned small language models (SLMs) can outperform LLMs on certain tasks and be deployed locally to mitigate privacy concerns, they underperform on more complex tasks such as text-to-SQL translation. In this work, we introduce MaskSQL, a text-to-SQL framework that utilizes abstraction as a privacy protection mechanism to mask sensitive information in LLM prompts. Unlike redaction, which removes content entirely, or generalization, which broadens tokens, abstraction retains essential information while discarding unnecessary details, striking an effective privacy–utility balance for the text-to-SQL task. Moreover, by providing mechanisms to control the privacy-utility tradeoff, MaskSQL facilitates adoption across a broader range of use cases. Our experimental results show that MaskSQL outperforms leading SLM-based text-to-SQL models and achieves performance approaching state-of-the-art LLM-based models, while preserving privacy. Our code is available at https://github.com/sepideh-abedini/MaskSQL.

1 Introduction

Structured databases are central to applications across science, business, healthcare, and government. However, retrieving information from these systems typically requires knowledge of Structured Query Language (SQL), creating a steep barrier for non-technical users. The text-to-SQL task bridges this gap by translating natural language (NL) questions into executable SQL queries, allowing users to interact with databases intuitively and without specialized expertise.

Recent advances in language models (LMs) have significantly improved the accuracy and availability of text-to-SQL solutions, enabling deployment across diverse domains and schemas. For example, the most performant approaches on the text-to-SQL benchmarks Spider [54], BIRD[26], and Spider 2.0 [22] rely on LMs as their backbone for SQL generation. An LM's performance typically scales with parameter count [9], leading to a common distinction [12] between small LMs (SLMs, < 10B parameters), and large LMs (LLMs, 10s to 100s of billions of parameters) which require specialized infrastructure to run. While LLMs dominate text-to-SQL benchmarks, due to their increased hardware requirements they are typically accessed via remote APIs hosted by specialized inference providers. These hosted APIs are attractive to users as they largely eliminate infrastructure outlays, but also

5202 peS 03]RC.sc[2v95432.9052:viXra

MaskSQL：通过抽象保护基于大型语言模型的文本到 SQL 的隐私

Sepideh Abedini[1,2] Shubhankar Mohapatra[1] D. B. Emerson[2] Masoumeh Shafieinejad[2] Jesse C. Cresswell[3] Xi He[1,2] 滑铁卢大学[1] {sepideh.abedini,shubhankar.mohapatra,xi.he}@uwaterloo.ca 向量研究所[2] Layer 6 AI[3] {david.emerson, masoumeh}@vectorinstitute.ai jesse@layer6.ai

摘要

大型语言模型（LLMs）在需要推理的任务上表现出令人鼓舞的性能，例如文本到SQL的转换、代码生成和调试。然而，具有严格隐私要求的监管框架限制了它们在敏感系统中的整合。最先进的LLMs也是专有的、昂贵的并且资源密集，使得本地部署不切实际。因此，利用此类LLMs通常需要与第三方提供者共享数据，这引发了隐私担忧并存在违反法规的风险。虽然经过微调的小型语言模型（SLMs）在某些任务上可以超过LLMs的表现并能够本地部署以减轻隐私问题，但它们在更复杂的任务上（如文本到SQL的转换）性能较差。在本工作中，我们介绍了MaskSQL，一个文本到SQL的框架，它利用抽象作为隐私保护机制，在LLM提示中屏蔽敏感信息。与完全删除内容的编辑或概化不同，

1 引言

结构化数据库在科学、商业、医疗和政府的应用中具有核心地位。然而，从这些系统中检索信息通常需要了解结构化查询语言（SQL），这对非专业用户造成了很高的门槛。文本到 SQL（text-to-SQL）任务通过将自然语言（NL）问题转换为可执行的 SQL 查询来弥合这一差距，使用户能够直观地与数据库交互，而无需专业知识。

最近在语言模型（LM）方面的进展显著提高了文本到SQL解决方案的准确性和可用性，使其能够在各种不同领域和模式中部署。例如，在文本到SQL基准测试Spider [54]、BIRD[26]和Spider 2.0 [22]上性能最优的方法都依赖语言模型作为其SQL生成的基础。LM的性能通常随参数数量的增加而提升 [9]，这导致了小型LM（SLMs，约10B参数）和大型LM（LLMs，参数数量为数十亿到数百亿）的常见区分 [12]，大型LM需要专门的基础设施来运行。尽管LLM在文本到SQL基准测试中占主导地位，但由于其对硬件的需求较高，它们通常通过由专门推理提供商托管的远程API访问。这些托管API对用户具有吸引力，因为它们大大减少了基础设施支出，但也

introduce severe privacy concerns; passing data through third-party APIs exposes sensitive user and schema information to heightened privacy risks [35, 29, 18]. For instance, recent work demonstrated that databases used in a text-to-SQL system are vulnerable to schema inference, where an adversary can reconstruct proprietary schema details by probing the system with carefully crafted queries [21]. These criticisms could lead to stricter standards for text-to-SQL tasks in privacy laws such as GDPR in Europe, HIPAA in the USA, or PIPEDA in Canada.

To illustrate the dilemma, consider the text-to-SQL task shown in Example 1. The user has two options to generate this query: (i) send their data to a powerful but *untrusted* LLM hosted remotely, with the risk of exposing schema and personally identifiable information (PII); or (ii) rely on *trusted* SLMs hosted locally, which often fail to handle complex SQL constructs such as nested queries, window functions, or common table expressions (see Figure 3 in Appendix E). Notable downsides exist with either option.

Example 1. *Assume a database schema with three tables that contain sensitive information about patients in hospitals in New York:*

$$T_1 : \textit{Patients}(pid, name, hiv_status, diagnosis, treatment)$$
$$T_2 : \textit{Hospital}(hid, name, address)$$
$$T_3 : \textit{Admissions}(aid, pid, hid, date)$$

A doctor with minimal database experience wants to generate an SQL query for the question $\mathcal{Q}$: "How many patients did the New York Hospital admit with HIV status as positive?". The doctor sends this query along with the database schema $\mathcal{S}$ to an LLM hosted remotely to generate the corresponding SQL query $\mathcal{Y}$. Although the LLM provider lacks access to the database and cannot execute the generated query $\mathcal{Y}$, the prompt has disclosed sensitive information: the existence of a table named "Patients", a column named "hiv_status", and possible literal values, such as "positive".

In this work, we propose a third alternative: MaskSQL, a privacy-preserving text-to-SQL framework that combines the utility of LLMs with the trust guarantees of local processing using SLMs. MaskSQL achieves this through *prompt abstraction*, which systematically replaces sensitive schema elements (table names, column names, and cell values) with abstract symbols before sending a text-to-SQL prompt to a remote LLM. Upon receiving the LLM's output, the SQL query is reconstructed locally to restore the abstracted values to their original values and make the query valid and executable.

However, implementing abstraction for text-to-SQL presents unique challenges: (i) accurately identifying sensitive tokens in the NL question according to user-defined privacy policies; (ii) preserving the utility of both the NL question and database schema in the abstraction process for accurate SQL generation; and (iii) correcting errors introduced by abstraction noise. MaskSQL addresses these with a three-stage pipeline of abstraction, SQL generation, and SQL reconstruction. On a challenging subset of the BIRD benchmark, MaskSQL outperforms state-of-the-art SLM-based approaches in accuracy, while preserving the privacy of user input, unlike LLM-based approaches.

The main contributions of this work are as follows.

1. We formalize the problem of privacy-preserving text-to-SQL using prompt abstraction guided by user-defined privacy policies.
2. We introduce MaskSQL, a framework that safeguards privacy when using untrusted remote LLMs by abstracting sensitive information in the question and database schema.
3. We propose a policy-based abstraction mechanism that enables a privacy–utility tradeoff, allowing users to customize the level of abstraction according to their needs.
4. We empirically evaluate MaskSQL on 300 complex queries from the BIRD benchmark, showing that it surpasses trusted SLM-based methods in accuracy while mitigating schema and PII leakage.

2 Background

In this section, we follow [57] to formalize the text-to-SQL task and provide some requisite background on LMs. We define a database schema as a tuple $\mathcal{S} = (\mathcal{T}, \mathcal{C}, \Phi)$ where $\mathcal{T} = \{T_1, T_2, \ldots\}$ is the set of tables, $\mathcal{C} = \{C_1, C_2, \ldots\}$ is the set of columns across all tables, and $\Phi = \{\phi_1, \phi_2, \ldots\}$ is the set of database constraints that enforce links between tables, such as primary or foreign keys. Each C_i is also associated with a set of literal values, V_i, present in the database such that $V_i \subset \text{dom}(C_i)$, where $\text{dom}(C_i)$ is the domain of column C_i. For example, V_i could represent real numbers for a

引入严重的隐私问题；通过第三方 API 传递数据会使敏感的用户和架构信息面临更高的隐私风险 [35, 29, 18]。例如，最近的研究表明，用于文本到 SQL 系统的数据库容易受到架构推断的攻击，攻击者可以通过精心设计的查询来重建专有的架构细节 [21]。这些批评可能导致在欧洲的 GDPR、美国的 HIPAA 或加拿大的 PIPEDA 等隐私法律中，对文本到 SQL 任务制定更严格的标准。

为了说明这一困境，请考虑示例 1 中显示的文本到 SQL 任务。用户有两种生成此查询的选项：(i) 将他们的数据发送到远程托管的强大但 *untrusted* 的大语言模型（LLM），有暴露模式和个人可识别信息（PII）的风险；或者 (ii) 依赖本地托管的 *trusted* 小语言模型（SLM），它们通常无法处理复杂的 SQL 结构，如嵌套查询、窗口函数或公共表表达式（见附录 E 的图 3）。这两种选项都有显著的缺点。

在这项工作中，我们提出第三种方案：MaskSQL，一种保护隐私的文本到SQL框架，它结合了大语言模型的实用性与使用小语言模型进行本地处理的信任保证。MaskSQL 通过 *prompt abstraction* 实现这一点，该方法在将文本到SQL的提示发送到远程大语言模型之前，会系统地将敏感的模式元素（表名、列名和单元格值）替换为抽象符号。在接收到大语言模型的输出后，SQL查询将在本地重建，以将抽象值还原为原始值，使查询有效并可执行。

示例 1. *Assume a database schema with three tables that contain sensitive information about patients in hospitals in New York:*

T_1 : *Patients*$(pid, name, hiv_status,$: *Hospital*$(hid, name, address)$: *Visits*$(vid, pid, hid, date,$
$diagnosis, treatment)$ T_3
T_2

A doctor with minimal database experience wants to generate an SQL query for the question $\mathcal{Q}$: "How many patients did the New York Hospital admit with HIV status as positive?". The doctor sends this query along with the database schema $\mathcal{S}$ to an LLM hosted remotely to generate the corresponding SQL query $\mathcal{Y}$. Although the LLM provider lacks access to the database and cannot execute the generated query $\mathcal{Y}$, the prompt has disclosed sensitive information: the existence of a table named "Patients", a column named "hiv_status", and possible literal values, such as "positive".

然而，为文本到 SQL 实现抽象带来了独特的挑战：(i) 根据用户定义的隐私政策，准确识别自然语言问题中的敏感标记；(ii) 在抽象过程中保持自然语言问题和数据库模式的可用性，以生成准确的 SQL；以及 (iii) 修正抽象噪声引入的错误。MaskSQL 通过抽象、SQL 生成和 SQL 重建的三阶段流程解决了这些问题。在 BIRD 基准的一个具有挑战性的子集中，MaskSQL 在准确性上优于最先进的基于 SLM 的方法，同时保护了用户输入的隐私，这一点不同于基于 LLM 的方法。

本工作的主要贡献如下。

1. 我们使用由用户定义的隐私策略指导的提示抽象，将隐私保护的文本到SQL问题形式化。 2. 我们引入了MaskSQL，一个在使用不受信任的远程大型语言模型时保护隐私的框架，通过对问题和数据库模式中的敏感信息进行抽象。 3. 我们提出了一种基于策略的抽象机制，实现隐私与效用的权衡，使用户可以根据需求自定义抽象级别。 4. 我们在BIRD基准的300个复杂查询上对MaskSQL进行了实证评估，结果显示其在准确性上超过了基于可信SLM的方法，同时减轻了模式和个人身份信息泄露。

2 背景

在本节中，我们遵循[57]来形式化文本到SQL的任务，并提供一些关于语言模型的必要背景。我们将数据库模式定义为一个元组$\mathcal{S} = (\mathcal{T}, \mathcal{C}, \Phi)$，其中$\mathcal{T} = \{T_1, T_2, \ldots\}$是表的集合，$\mathcal{C} = \{C_1, C_2, \ldots\}$是所有表的列的集合，$\Phi = \{\phi_1, \phi_2, \ldots\}$是数据库约束的集合，用于在表之间建立连接，例如主键或外键。每个C_i还关联一组在数据库中存在的字面值V_i，使得$V_i \subset$ dom(C_i)，其中dom(C_i)是列C_i的域。例如，V_i可以表示实数用于一个

numerical column, or textual categories for a categorical column. Herein, we consider access only to the schema $\mathcal{S}$ and not the values $V_i \in \mathcal{V}$, where $\mathcal{V}$ is the set of all literal values in the database.

Text-to-SQL Task Formulation. Text-to-SQL translation aims to map an NL question to its corresponding executable SQL code with respect to a given database schema [40]. Formally, given a natural language question $\mathcal{Q}$ and a database schema $\mathcal{S}$, the task is to generate a SQL query $\mathcal{Y}$, such that $\mathcal{Y}$ is executable and accurately represents the intent of $\mathcal{Q}$. $\mathcal{Q}$ is represented as a sequence of tokens such that $\mathcal{Q} = w_1, \ldots, w_n$ for $w_i \in \mathbb{W}$, where $\mathbb{W}$ is the vocabulary of permissible tokens (we use the English language vocabulary).

Language Models. Language Models are neural models trained on large-scale corpora of natural language and structured text. They are often informally classified into two types based on the total number of parameters. LLMs have a very high parameter count (tens to hundreds of billions) which requires multiple GPUs or specialized hardware to run, and, hence, are usually accessed through a third-party service. In contrast, SLMs use fewer parameters (less than ten billion) and can typically be run on a single GPU, enabling them to be hosted locally. Due to the difference in model complexity, SLMs have reduced utility and ability to reason compared with LLMs [50].

LMs are used by providing a prompt to specify the task to be performed. Hosted LLMs require the prompt to be passed through an API to the remote infrastructure. Once this happens, the owner loses control over how their data is processed, used, or stored, which is a major concern when there is an obligation to protect that data. Compliance with relevant laws and regulations around data privacy may preclude the use of hosted LLMs due to this loss of control. Thus, there is a need for solutions that balance the performance of LLMs with the increased control and privacy that SLMs bring.

In the context of this paper, we use LMs for the generation of SQL queries from NL questions. Given question $\mathcal{Q}$ and schema $\mathcal{S}$, we form a prompt $P(\mathcal{Q}, \mathcal{S})$ which is input to an LM denoted by f_{LM}. The text-to-SQL task is represented as $\mathcal{Y} = f_{\text{LM}}(P(\mathcal{Q}, \mathcal{S}))$, where $\mathcal{Y}$ is the generated SQL query.

3 Related Work

Text-to-SQL translation has been extensively studied in both the NLP and database communities. Earlier methods relied on rule-based systems and named-entity recognition (NER) [3, 41], followed by neural approaches using LSTMs [48, 58, 51, 53] and transformers [23, 30]. More recently, prompt-based LLMs have emerged as the state-of-the-art [38, 13]. Comprehensive comparisons of these techniques are provided on benchmarks including Spider [54], BIRD [26], Spider 2.0 [22], and in recent surveys [59, 44, 10]. An alternative line of work focuses on locally served SLMs. Several methods [39, 25, 14, 43] improve SLM performance via fine-tuning. While using local SLMs eliminates the privacy concerns of sharing data with third parties, they consistently underperform on complex queries requiring stronger reasoning [43]. Our approach follows a hybrid strategy, combining SLMs and LLMs, and we evaluate it against some state-of-the-art models reported on the BIRD leaderboard.[1]

A parallel line of work emphasizes privacy-preserving LLM inference, as user prompts often contain sensitive information. Cryptographic approaches include homomorphic encryption (HE) [6] and multi-party computation (MPC) [2], which protect user data during inference but incur significant computational overhead. Differential privacy (DP) has also been applied to preserve training privacy [33, 19, 17, 34] or inference privacy using noisy embeddings [32, 11] and token substitutions [47, 55]. However, such methods either do not operate well at the scale of LLMs, or degrade model utility [52]—a critical limitation for text-to-SQL, where precise information retrieval is required. In contrast, we propose a practical privacy protection approach, leveraging local SLMs and prompt abstraction to minimize the exposure of sensitive data while preserving semantic fidelity.

Recent work has explored prompt sanitization for LLM inference privacy [7, 56, 27, 20, 8]. For example, Portcullis [56] employs NER to detect sensitive entities, while PP-TS [20] and Pr$\varepsilon\varepsilon$mpt [8] sanitize inputs by replacing PII with contextually appropriate surrogates. HaS [7] and Papillon [27] rely on fine-tuned local models for anonymization. Related to these are generalization-based methods [42, 5, 4], which replace values with general terms to obscure specifics. Unlike these general-purpose techniques, the abstraction process we apply for text-to-SQL consistently preserves the alignment between the question's logic and the database schema.

[1] https://bird-bench.github.io/

数值列，或分类列的文本类别。在此，我们仅考虑访问模式 $\mathcal{S}$ 而不访问值 $V_i \in \mathcal{V}$，其中 $\mathcal{V}$ 是数据库中所有字面值的集合。

文本到 SQL 任务的表述。文本到 SQL 翻译旨在将自然语言问题映射到其对应的可执行 SQL 代码，并且遵循给定的数据库模式 [40]。形式上，给定自然语言问题 $\mathcal{Q}$ 和数据库模式 $\mathcal{S}$，任务是生成 SQL 查询 $\mathcal{Y}$，使得 $\mathcal{Y}$ 可执行并准确表达 $\mathcal{Q}$ 的意图。$\mathcal{Q}$ 被表示为一个标记序列，使得 $\mathcal{Q} = w_1, \ldots, w_n$ 对于 $w_i \in \mathbb{W}$，其中 $\mathbb{W}$ 是允许的标记词汇表（我们使用英语词汇表）。

语言模型。语言模型是基于大规模自然语言和结构化文本语料训练的神经模型。它们通常根据参数总数非正式地分为两类。大型语言模型（LLM）具有非常高的参数量（数十亿到数百亿），运行它们需要多块GPU或专用硬件，因此通常通过第三方服务访问。相比之下，小型语言模型（SLM）使用的参数较少（少于十亿），通常可以在单块GPU上运行，从而可以本地托管。由于模型复杂度的差异，SLM相比LLM在实用性和推理能力上都有所下降[50]。

语言模型（LMs）通过提供提示来指定要执行的任务。托管的语言模型（LLMs）需要通过 API 将提示传递到远程基础设施。一旦发生这种情况，所有者就会失去对其数据如何被处理、使用或存储的控制，而当有义务保护这些数据时，这就是一个主要的担忧。由于这种控制权的丧失，遵守有关数据隐私的相关法律法规可能会排除使用托管的LLMs。因此，需要寻找能够在语言模型性能与自有语言模型（SLMs）所带来的增强控制和隐私之间取得平衡的解决方案。

在本文的背景下，我们使用语言模型（LM）从自然语言问题生成 SQL 查询。给定问题 $\mathcal{Q}$ 和模式 $\mathcal{S}$，我们形成一个提示 $P(\mathcal{Q}, \mathcal{S})$，该提示被输入到表示为 f_{LM} 的语言模型中。文本到 SQL 的任务表示为 $\mathcal{Y} = f_{\text{LM}}(P(\mathcal{Q}, \mathcal{S}))$，其中 $\mathcal{Y}$ 是生成的 SQL 查询。

3 相关工作

文本到SQL的翻译已经在自然语言处理（NLP）和数据库社区中得到了广泛研究。早期方法依赖基于规则的系统和命名实体识别（NER）[3, 41]，随后出现了使用LSTM [48, 58, 51, 53] 和 Transformer [23, 30] 的神经方法。最近，基于提示的大型语言模型（LLM）已经成为最先进的方法[38, 13]。对这些技术的全面比较可在包括Spider [54]、BIRD [26]、Spider 2.0 [22] 的基准测试以及最近的综述[59, 44, 10]中找到。另一类工作关注本地部署的专用语言模型（SLM）。若干方法[39, 25, 14, 43] 通过微调提升了SLM的性能。虽然使用本地SLM消除了与第三方共享数据的隐私问题，但在需要更强推理能力的复杂查询中，它们的表现仍然不如人意[43]。我们的方法采用混合策略，结合SLM和LLM，并在BIRD排行榜上与一些最先进的模型进行评估。

一条平行的研究路线强调隐私保护的大型语言模型（LLM）推理，因为用户的提示通常包含敏感信息。加密方法包括同态加密（HE）[6] 和多方计算（MPC）[2]，这些方法在推理过程中保护用户数据，但会带来显著的计算开销。差分隐私（DP）也被用于保护训练隐私 [33, 19, 17, 34] 或在推理阶段通过引入噪声嵌入 [32, 11] 和词元替换 [47, 55] 来保护隐私。然而，这些方法要么在大型语言模型的规模上表现不佳，要么会降低模型效用 [52]——这对需要精确信息检索的文本到 SQL 应用来说是一个关键限制。相比之下，我们提出了一种实用的隐私保护方法，利用本地小型语言模型（SLM）和提示抽象来最小化敏感数据的暴露，同时保持语义的准确性。

最近的研究探索了用于大语言模型推理隐私的提示清理方法 [7, 56, 27, 20, 8]。例如，Portcullis [56] 利用命名实体识别（NER）来检测敏感实体，而 PP-TS [20] 和 Pr$\varepsilon\varepsilon$mpt [8] 通过用上下文合适的替代信息替换个人可识别信息（PII）来清理输入。HaS [7] 和 Papillon [27] 则依赖微调的本地模型进行匿名化。这些方法相关的还有基于泛化的方法 [42, 5, 4]，它们用概括性术语替换具体值以模糊细节。不同于这些通用技术，我们在文本到 SQL 的抽象过程中始终保持问题逻辑与数据库模式之间的一致性。

[1] https://bird-bench.github.io/

4 Problem Statement

Consider a user with limited local compute resources who, given their database schema $\mathcal{S}$ and an NL question $\mathcal{Q}$, wants to generate a SQL query $\mathcal{Y}$ that performs the task described in $\mathcal{Q}$. The user could be, for example, a financial institution or a hospital that holds sensitive information about individuals and is prohibited from sharing that information with untrusted third parties. As such, we assume that any remotely hosted LLM is untrusted [49]. Specifically, we consider a privacy policy Ψ that defines what information in $\mathcal{S}$, $\mathcal{Q}$, or the database values $\mathcal{V}$ is considered sensitive. A privacy policy is determined by the user according to their needs and may contain all or a subset of the following information.

- **Table and Column Names:** Any table or columns names from the sets of tables $\mathcal{T}$ and columns $\mathcal{C}$ defined in $\mathcal{S}$, including words or terms in $\mathcal{Q}$ that refer such tables or columns.
- **Literal Values:** Any words or terms in $\mathcal{Q}$ that might refer to any cell value from the set $\mathcal{V}$ in the database.

By default, we use the full policy Ψ_F which includes the entire schema $\mathcal{S}$ and all values $\mathcal{V}$, as sensitive. However, less restrictive policies can also be defined, depending on the user's requirements. The problem of privacy-preserving text-to-SQL is to leverage a LM to accurately generate an SQL query $\mathcal{Y}$ that correctly implements the intent of $\mathcal{Q}$ without exposing any information defined in the policy Ψ.

5 MaskSQL: Privacy-Preserving Text-to-SQL Generation

Our proposed approach, MaskSQL, leverages LLM capabilities while protecting sensitive information. We identify that, for a model to generate a correct SQL query $\mathcal{Y}$, the essential information in the prompt is the mapping between the terms used in the NL question $\mathcal{Q}$ and entities in the database schema $\mathcal{S}$. Specific names or values are not crucial for the structure and syntax of the generated query. Thus, table names, column names, and literal values can be abstracted away in the prompt and later restored by using a bijective mapping between the original and abstract tokens. Consider Example 1: $\mathcal{Q}$: "*How many patients did the New York Hospital admit with HIV status as positive?*". An abstracted query, "*How many T_1 did the V_1 T_3 with C_3 as V_2?*", paired with an appropriately abstracted schema retains all information needed to generate $\mathcal{Y}$.

There are multiple challenges with respect to implementing such an abstraction approach in practice. First, the tokens in $\mathcal{Q}$ must be accurately linked to the corresponding elements in $\mathcal{S}$. This is difficult because a non-technical user may enter tokens in $\mathcal{Q}$ that do not exactly match those used in $\mathcal{S}$. For instance, the user may write the token *admit* which must be linked to the table named *Admissions* based on the semantic context. Second, the abstracted prompt is processed by an LLM, which generates a similarly abstracted SQL query. The abstracted SQL query then needs to be reliably mapped back to its concrete form. This step needs to be performed accurately, as incorrect mapping can lead to syntax errors (e.g. joins applied with incorrect column names). Third, $\mathcal{Y}$ may contain minor errors or issues (e.g., using the string value of *positive* instead of the numeric value of 1) that must be corrected. These errors can occur due to noisy translation during the process and must be corrected for error-free query execution.

MaskSQL addresses these challenges within a three-stage pipeline: *Abstraction*, *SQL Generation*, and *SQL Reconstruction*. Altogether, this pipeline safeguards user privacy while leveraging the capabilities of LLMs. An overview of the pipeline is shown in Figure 1. In the first stage, $\mathcal{Q}$, $\mathcal{S}$, and Ψ are passed to the *Abstraction* engine which produces abstracted representations $\mathcal{Q}'$ and $\mathcal{S}'$. In the second stage, these abstracted inputs are combined into a prompt $P(\mathcal{Q}', \mathcal{S}')$ and provided to the *SQL Generation* engine, which prompts a remotely hosted LLM and returns an abstracted SQL query $\mathcal{Y}'$. In the final stage, the *SQL Reconstruction* engine maps $\mathcal{Y}'$ back to its concrete form $\mathcal{Y}$, applies a final round of corrections to fix any remaining errors, and outputs the executable SQL query. Figure 4 in Appendix E shows $\mathcal{Q}'$, $\mathcal{S}'$, and $\mathcal{Y}'$ under the full policy, Ψ_F, for Example 1. The remainder of this section describes each stage in detail.

4 问题陈述

考虑一个本地计算资源有限的用户，已知其数据库模式 $\mathcal{S}$ 和自然语言问题 $\mathcal{Q}$，希望生成一个 SQL 查询 $\mathcal{Y}$ 来执行 $\mathcal{Q}$ 中描述的任务。例如，该用户可以是持有个人敏感信息并被禁止与不受信任第三方共享这些信息的金融机构或医院。因此，我们假设任何远程托管的语言模型都是不受信任的 [49]。具体来说，我们考虑一个隐私政策 Ψ，其定义了 $\mathcal{S}$、$\mathcal{Q}$ 或数据库值 $\mathcal{V}$ 中哪些信息被视为敏感。隐私政策由用户根据其需求确定，并可能包含以下信息的全部或子集。

- 表和列名称：由 $\mathcal{S}$ 中定义的表集合 $\mathcal{T}$和列 $\mathcal{C}$ 中的任何表或列名称，包括 $\mathcal{Q}$ 中指这些表或列的单词或术语。
- 字面值：$\mathcal{Q}$ 中的任何单词或术语，可能指数据库中集合 $\mathcal{V}$ 的任何单元格值。

默认情况下，我们使用完整策略 Ψ_F，其中将整个模式 $\mathcal{S}$ 和所有值 $\mathcal{V}$ 视为敏感信息。然而，也可以根据用户的需求定义限制较少的策略。隐私保护的文本到 SQL 问题是利用语言模型准确生成一个 SQL 查询 $\mathcal{Y}$，该查询能够正确实现 $\mathcal{Q}$ 的意图，同时不泄露策略 Ψ 中定义的任何信息。

5 MaskSQL：隐私保护的文本到SQL生成

我们提出的方法 MaskSQL，在利用大型语言模型（LLM）能力的同时保护敏感信息。我们发现，对于模型生成正确的 SQL 查询 $\mathcal{Y}$，提示中关键信息是自然语言问题 $\mathcal{Q}$ 中使用的术语与数据库模式 $\mathcal{S}$ 中实体的映射。具体的名称或数值对于生成的查询的结构和语法并不关键。因此，表名、列名和字面值可以在提示中抽象化，然后通过使用原始标记和抽象标记之间的一一映射进行恢复。考虑示例 1：$\mathcal{Q}$：“*How many patients did the New York Hospital admit with HIV status as positive?*”。一个抽象化的查询，“*How many* T_1 *did the* V_1 T_3 *with* C_3 *as* V_2*?*”，配合适当抽象化的模式，保留了生成 $\mathcal{Y}$ 所需的所有信息。

在实际实施这种抽象方法时存在多个挑战。首先，$\mathcal{Q}$ 中的标记必须准确地链接到 $\mathcal{S}$ 中的相应元素。这很困难，因为非技术用户可能在 $\mathcal{Q}$ 中输入的标记并不完全匹配 $\mathcal{S}$ 中使用的标记。例如，用户可能写下标记 *admit*，必须根据语义上下文将其链接到名为 *Admissions* 的表。其次，抽象化的提示由大语言模型（LLM）处理，生成类似抽象化的 SQL 查询。然后，抽象化的 SQL 查询需要可靠地映射回其具体形式。这一步需要准确执行，因为映射错误可能导致语法错误（例如使用错误的列名进行连接）。第三，$\mathcal{Y}$ 可能包含一些小错误或问题（例如使用 *positive* 的字符串值而不是数字值 1），必须进行修正。这些错误可能由于过程中出现噪声翻译而发生，必须予以纠正。

MaskSQL 在一个三阶段的流程中应对这些挑战：*Abstraction*、*SQL Generation* 和 *SQL Reconstruction*。总体来说，这个流程在利用大型语言模型（LLM）能力的同时保护用户隐私。流程概览如图 1 所示。在第一阶段，$\mathcal{Q}$、$\mathcal{S}$ 和 Ψ 被传递到 *Abstraction* 引擎，生成抽象表示 $\mathcal{Q}'$ 和 $\mathcal{S}'$。在第二阶段，这些抽象输入被组合成提示 $P(\mathcal{Q}', \mathcal{S}')$ 并提供给 *SQL Generation* 引擎，该引擎提示远程托管的 LLM 并返回抽象的 SQL 查询 $\mathcal{Y}'$。在最后阶段，*SQL Reconstruction* 引擎将 $\mathcal{Y}'$ 映射回其具体形式 $\mathcal{Y}$，进行最后一轮修正以修复剩余错误，并输出可执行的 SQL 查询。附录 E 中的图 4 显示了示例 1 下全策略 Ψ_F 的 $\mathcal{Q}'$、$\mathcal{S}'$ 和 $\mathcal{Y}'$。本节剩余部分将详细描述每个阶段。

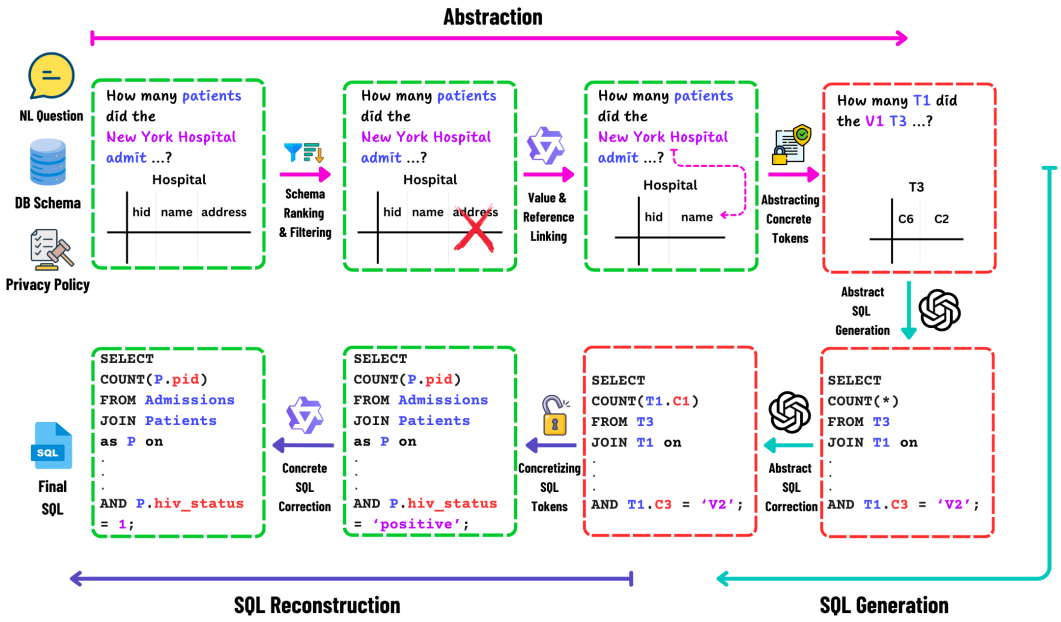


Figure 1: MaskSQL pipeline. Green dashed boxes delineate text and schema information contained in the *"trusted environment"*, while red boxes denote those exposed to *"untrusted third parties"*.

5.1 Abstraction

The first stage of the pipeline generates abstract versions of both $\mathcal{Q}$ and $\mathcal{S}$. This process consists of three main steps.

1. Ranking and filtering entire elements of $\mathcal{S}$ to drop irrelevant tables and columns based on $\mathcal{Q}$.
2. Identifying mappings between sensitive terms in $\mathcal{Q}$ and the retained schema elements using a locally hosted SLM.
3. Replacing the identified terms in $\mathcal{Q}$ and the retained schema elements with abstract identifiers.

The bijective masking procedure, combined with SLM-based schema linking, provides an effective approach for abstracting a text-to-SQL query. We detail each of these steps below.

Schema Ranking and Filtering. Real-world databases often include hundreds of tables and columns, which can overwhelm LMs with irrelevant context. To mitigate this, we use a local cross-encoder model, following the methodology of RESDSQL [24], to rank and filter schema elements based on their relevance to $\mathcal{Q}$. Specifically, RESDSQL employs a RoBERTa-based cross-encoder [28] to compute contextual embeddings for both $\mathcal{Q}$ and $\mathcal{S}$. These embeddings are pooled using a Bidirectional Long Short-Term Memory [16] and scored via Multi-Layer Perceptron modules to estimate the relevance of each table and column. Based on these scores, the top-k tables and their corresponding top-j columns are retained. This strategy is integrated into the pipeline, as keeping too few elements may result in missing relevant matches, while too many can introduce noise and increase computational overhead. As shown in Appendix C, schema filtering has a significant impact on both accuracy and efficiency. In our experiments, retaining $k = 4$ tables and $j = 5$ columns per table yields strong performance, though these parameters can be adjusted based on specific use cases. The output of this step is a ranked and filtered list of relevant schema elements, which are then passed to subsequent stages.

Value and Reference Linking. This step takes as input the filtered list of schema elements from the previous step and $\mathcal{Q}$, and constructs a mapping between the two using a local SLM. This process is performed in three sub-steps. First, the SLM is prompted to identify tokens $w_i \in \mathcal{Q}$ that may correspond to values in the database, $\mathcal{V}$. In Example 1, the tokens *New York Hospital* and *positive* are identified. In the second step, the SLM is prompted to map the identified values to their corresponding columns and tables, from $\mathcal{C}$ and $\mathcal{T}$ in $\mathcal{S}$, respectively. For Example 1, *New York Hospital* is mapped to *Hospital.name* and *positive* is mapped to *Patients.hiv_status*. In the final step, the SLM is prompted

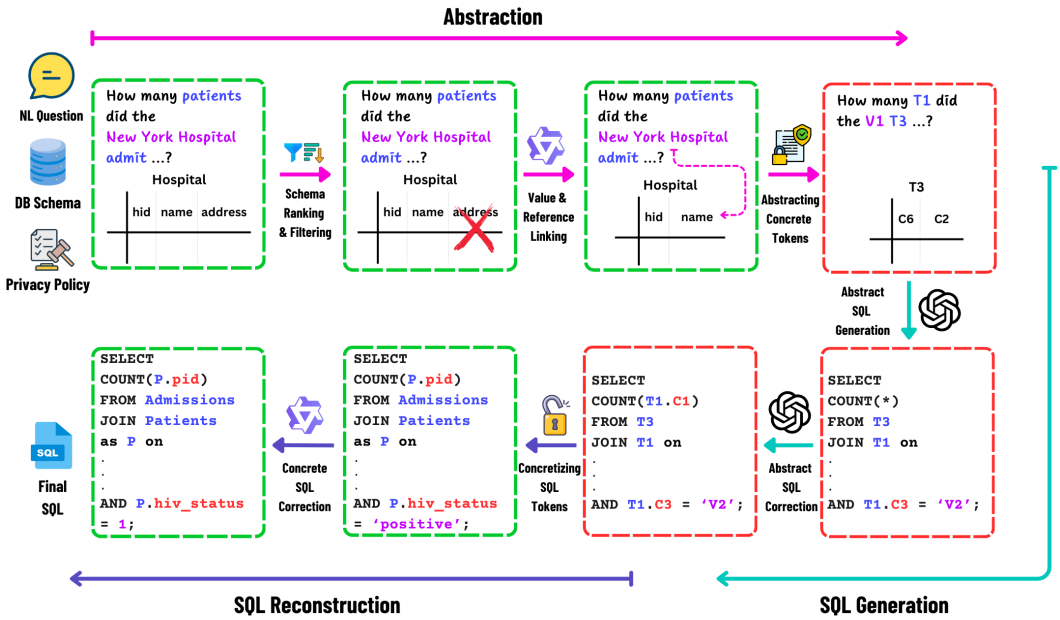


图 1：MaskSQL 流程。绿色虚线框表示 *"trusted environment"* 中包含的文本和模式信息，而红色框表示暴露给 *"untrusted third parties"* 的信息。

5.1 抽象

管道的第一阶段会生成 $\mathcal{Q}$ 和 $\mathcal{S}$ 的抽象版本。这个过程包括三个主要步骤。

1. 对 $\mathcal{S}$ 的整个元素进行排序和过滤，以根据 $\mathcal{Q}$ 删除无关的表和列。 2. 使用本地托管的 SLM，识别 $\mathcal{Q}$ 中敏感术语与保留的模式元素之间的映射。 3. 用抽象标识符替换 $\mathcal{Q}$ 中识别出的术语及保留的模式元素。

双射掩码程序结合基于SLM的模式连接，为抽象文本到SQL查询提供了一种有效的方法。我们在下面详细说明每个步骤。

模式排序与过滤。现实世界的数据库通常包含数百个表和列，这可能会让语言模型被无关的上下文淹没。为缓解这一问题，我们使用一个本地的交叉编码模型，遵循 RESDSQL [24] 的方法，根据与 $\mathcal{Q}$ 的相关性对模式元素进行排序和过滤。具体来说，RESDSQL 使用基于 RoBERTa 的交叉编码器 [28] 来计算 $\mathcal{Q}$ 和 $\mathcal{S}$ 的上下文嵌入。这些嵌入通过双向长短期记忆网络（BiLSTM）［16］进行汇聚，并通过多层感知机模块评分，以评估每个表和列的相关性。根据这些评分，保留前 k 个表及其对应的前 j 列。该策略被整合到流程中，因为保留元素过少可能导致遗漏相关匹配，而过多则会引入噪声并增加计算开销。如附录 C 所示，模式过滤对准确性和效率都有显著影响。

值与引用链接。此步骤以上一步筛选出的模式元素列表和$\mathcal{Q}$作为输入，并使用本地SLM构建两者之间的映射。该过程分为三个子步骤。首先，提示SLM识别可能对应数据库中值的标记$w_i \in \mathcal{Q}$，$\mathcal{V}$。在示例1中，识别出标记*New York Hospital*和*positive*。在第二步中，提示SLM将识别出的值映射到相应的列和表，分别来自$\mathcal{C}$和$\mathcal{T}$中的$\mathcal{S}$。对于示例1，*New York Hospital*映射到*Hospital.name*，*positive*映射到*Patients.hiv_status*。在最后一步，提示SLM

to identify any remaining tokens $w_i \in \mathcal{Q}$ that may reference any column or table names in the filtered schema elements. In Example 1, tokens *patients*, *admit*, and *HIV status* are mapped to the tables *Patients* and *Admissions*, and column *Patients.hiv_status*, respectively. These mappings are then passed to the next step.

The importance of value and reference linking is two-fold. To protect privacy and abstract away sensitive tokens, it is essential to generate an accurate and complete linking; any token not identified as a value or reference remains unmasked and, therefore, may be exposed. In addition, the linking map must be generated precisely to preserve reference information during the abstraction process. Also, prior work shows that SLMs achieve accuracy comparable to LLMs for schema linking tasks [31], making them sufficient for this stage.

Abstracting Concrete Tokens. The final step generates the abstracted NL question $\mathcal{Q}'$ and the schema $\mathcal{S}'$. To generate $\mathcal{S}'$, each table, column, and value specified in Ψ and retained after the initial schema filtering step is assigned an abstract symbol. For example, table names are mapped to symbols like T_i, and columns to C_i. Tokens that are not included in the privacy policy Ψ remain unchanged. The mapping from $\mathcal{S}$ to $\mathcal{S}'$ is stored as a symbol lookup table, which is later used for both masking and reconstruction. Next, using this symbol table and the linking map from the previous step, all references to tables and columns in $\mathcal{Q}$ are replaced with their corresponding abstract symbols. In Example 1, the table name *Patients* is represented by the symbol T_1, column *Patients.pid* by symbol C_1, and column *Patients.name* by symbol C_2, and so on.

Finally, each literal value in $\mathcal{Q}$ is also replaced with a unique abstract symbol of the form V_i. As shown in Figure 1, the token *New York Hospital* is mapped to V_1. An additional sentence is also appended to the question to specify the column associated with the value. In Example, 1, the sentence: "V_1 is a value of the column C_7." where C_7 is the abstracted symbol for column *Hospital.name*, is appended to the abstract question. This additional context helps the LLM understand the alignment between values and schema elements without exposing any concrete tokens.

A key point is that, once the schema linking map is generated, abstraction reduces to simple text substitution. This method preserves the reference information in the question while allowing for accurate abstraction inversion of an LLM's output.

5.2 SQL Generation

Using the abstracted question $\mathcal{Q}'$ and database schema $\mathcal{S}'$ produced in the previous steps, a remotely hosted LLM is prompted to generate the corresponding abstracted SQL query $\mathcal{Y}'$. The generation prompt used in the experiments is provided in Appendix F.1. This prompt contains only $\mathcal{Q}'$ and $\mathcal{S}'$, with no additional sensitive information. Since the LLM only sees abstract symbols, the generated SQL query is also expected to follow the abstract form. An example of such an abstract SQL query is shown in Figure 1. To address minor errors in the generated abstract SQL, a self-correction mechanism, commonly used in text-to-SQL pipelines, is employed [38]. In this step, the LLM is prompted with $\mathcal{Q}'$, $\mathcal{S}'$, and $\mathcal{Y}'$ and is instructed to identify and correct any potential issues. The prompt used for this step is included in Appendix F.2.

5.3 SQL Reconstruction

In the final stage, the abstract SQL query $\mathcal{Y}'$ is mapped back to its concrete form $\mathcal{Y}$. Using the symbol lookup table created during abstraction, all abstract symbols are replaced with their corresponding concrete values. The result is an executable SQL query free of abstract identifiers. To further improve accuracy, an additional self-correction step is applied using a local SLM. The concretized SQL query is executed on the target database, and its execution result, along with $\mathcal{Q}$, $\mathcal{S}$, and the concretized query $\mathcal{Y}$, is passed to the SLM to correct any remaining errors. The full prompt for this step is provided in Appendix F.3.

6 Experiments

In this section, experimental results are reported comparing MaskSQL with several state-of-the-art text-to-SQL frameworks. We use the BIRD dataset, a widely used benchmark consisting of NL questions paired with ground-truth SQL queries, along with the corresponding databases that enable

识别在过滤后的架构元素中可能引用任何列或表名称的剩余标记 $w_i \in \mathcal{Q}$。在示例 1 中，标记 *patients*、*admit* 和 *HIV status* 分别映射到表 *Patients* 和 *Admissions* 以及列 *Patients.hiv_status*。这些映射随后会传递到下一步。

值和引用链接的重要性有两个方面。为了保护隐私并抽象敏感的令牌，生成准确且完整的链接是必不可少的；任何未被识别为值或引用的令牌将保持未掩码状态，因此可能被暴露。此外，必须精确生成链接映射以在抽象过程中保留引用信息。同样，先前的研究表明，SLM 在模式链接任务中的准确性可与 LLM 相媲美 [31]，因此它们在此阶段是足够的。

抽象具体标记。最后一步生成抽象后的自然语言问题 $\mathcal{Q}'$ 和模式 $\mathcal{S}'$。为了生成 $\mathcal{S}'$，在初步模式过滤步骤后保留的每个表格、列和在 Ψ 中指定的值都会被分配一个抽象符号。例如，表名被映射为诸如 T_i 的符号，列映射为 C_i。未包含在隐私政策 Ψ 中的标记保持不变。从 $\mathcal{S}$ 到 $\mathcal{S}'$ 的映射被存储为符号查找表，稍后用于屏蔽和重建。接下来，使用此符号表和上一步的链接映射，$\mathcal{Q}$ 中所有对表格和列的引用都被替换为相应的抽象符号。在示例 1 中，表名 *Patients* 用符号 T_1 表示，列 *Patients.pid* 用符号 C_1 表示，列 *Patients.name* 用符号 C_2 表示，依此类推。

最后，$\mathcal{Q}$ 中的每个字面值也被替换为形式为 V_i 的唯一抽象符号。如图 1 所示，符号 *New York Hospital* 被映射到 V_1。此外，还在问题中附加了一句话以指定与该值关联的列。在示例 1 中，句子：“V_1 是列 C_7 的一个值。”其中 C_7 是列 *Hospital.name* 的抽象符号，被附加到抽象问题中。这一附加的上下文有助于大语言模型理解值与模式元素之间的对应关系，而不会暴露任何具体的标记。

一个关键点是，一旦生成了模式链接映射，抽象就简化为简单的文本替换。这种方法在允许对大语言模型输出进行准确抽象反演的同时，保留了问题中的参考信息。

5.2 SQL 生成

使用在前几步中生成的抽象化问题 $\mathcal{Q}'$ 和数据库模式 $\mathcal{S}'$，远程托管的 LLM 被提示生成相应的抽象化 SQL 查询 $\mathcal{Y}'$。实验中使用的生成提示位于附录 F.1。该提示仅包含 $\mathcal{Q}'$ 和 $\mathcal{S}'$，没有额外的敏感信息。由于 LLM 只能看到抽象符号，生成的 SQL 查询也预计会遵循抽象形式。此类抽象 SQL 查询的示例如图 1 所示。为了处理生成的抽象 SQL 中的轻微错误，采用了一种自我纠正机制，该机制在文本到 SQL 的流程中常用［38］。在此步骤中，LLM 被提示 $\mathcal{Q}'$、$\mathcal{S}'$ 和 $\mathcal{Y}'$，并被指示识别和纠正任何潜在问题。用于此步骤的提示包含在附录 F.2。

5.3 SQL 重建

在最后阶段，抽象的 SQL 查询 $\mathcal{Y}'$ 被映射回其具体形式 $\mathcal{Y}$。使用在抽象过程中创建的符号查找表，所有抽象符号都被其对应的具体值替换。结果是一个可执行的、不含抽象标识符的 SQL 查询。为了进一步提高准确性，会使用本地 SLM 进行额外的自我校正步骤。具体化后的 SQL 查询会在目标数据库上执行，其执行结果以及 $\mathcal{Q}$、$\mathcal{S}$ 和具体化的查询 $\mathcal{Y}$ 都会传递给 SLM，以纠正任何剩余的错误。本阶段的完整提示在附录 F.3 中提供。

6 个实验

在本节中，报告了实验结果，比较了 MaskSQL 与几种最先进的文本到 SQL 框架。我们使用 BIRD 数据集，这是一个广泛使用的基准，包含自然语言问题与真实 SQL 查询的配对，以及对应的数据库，这些数据库使得……

execution-based evaluation of generated queries [26]. To demonstrate the gap between state-of-the-art LLM- and SLM-based text-to-SQL models on more complex queries, a challenging subset of 300 entries is selected from the BIRD development split. In particular, we selected examples that involve complex SQL patterns, such as nested queries, set operations (e.g., INTERSECT), and multiple joins. To generate ground-truth data for privacy measurements, GPT-4.1 [37] is used to annotate tokens that should be abstracted, followed by human review and correction. For MaskSQL, Qwen-2.5-7B-Instruct [46] serves as the trusted local SLM, and GPT-4.1 is used as the untrusted LLM for generating SQL queries from abstracted prompts.

We evaluate MaskSQL under two privacy policies.

- Ψ_F: The full privacy policy, defined in Section 4, where the entire database schema and associated values are considered sensitive.
- Ψ_C: A category-based policy where only tokens related to concepts of person names, occupations, and locations are abstracted. The formal definition is provided in Appendix A.

6.1 Baselines

We compare MaskSQL against several baselines, including three state-of-the-art text-to-SQL frameworks and direct LM prompting. The direct prompting baseline applies a simple few-shot prompt that asks the LM to generate a SQL query based on the given NL question and database schema. DAIL-SQL [13], which holds the top position on the Spider benchmark [54] for open-source solutions at the time of writing, builds on few-shot prompting with additional strategies to improve SQL generation accuracy. DIN-SQL [38], which holds the second-best ranking among open-source frameworks, decomposes text-to-SQL translation into smaller tasks to improve the accuracy. For direct prompting, DAIL-SQL, and DIN-SQL, we consider both a trusted setting, where Qwen2.5-7B-Instruct is used as the backbone LM and an untrusted setting, where GPT-4.1 is used. Finally, MSc-SQL [14] is designed specifically for SLM-only settings. It samples a few candidate SQL queries from different models and then selects the best candidate by prompting additional SLMs. For experiments leveraging MSc-SQL, we use the fine-tuned SLMs released by the authors based on Gemma-2-9B-it [45], Llama-3-8B [15], and Mistral-7B-Instruct-v0.2 [1].

6.2 Metrics

We evaluate each baseline on utility, efficiency, and privacy metrics. For utility evaluation, we use execution accuracy, as defined in the BIRD benchmark. Efficiency is measured by average token usage per query generation. For privacy, we define two metrics.

Masking Recall (MR): This metric is defined as the ratio of correctly abstracted tokens to the total number of ground-truth sensitive tokens in the NL question $\mathcal{Q}$. Higher values indicate more protection of the sensitive tokens, resulting in better privacy.

Re-identification Score (RI): This metric captures the proportion of abstracted tokens in the NL question $\mathcal{Q}'$ that cannot be re-identified by an adversary. Specifically, we prompt GPT-4.1 with the abstracted NL question $\mathcal{Q}'$ and schema $\mathcal{S}'$ and instruct it to infer the original tokens. The score is then computed as the ratio of the tokens that cannot be recovered by the LLM to the total number of abstract tokens in $\mathcal{Q}'$.

Formal definitions of these metrics are provided in Appendix B.

6.3 Results

Accuracy. Table 1 shows the execution accuracy across baselines. Untrusted LLM-based approaches perform significantly better than trusted SLM-based methods, which is expected given the additional capacity of LLMs. However, the direct use of LLMs exposes sensitive data to privacy risks. MaskSQL outperforms all trusted SLM-only approaches, achieving an execution accuracy of 55.66% under the full privacy policy, Ψ_F, which is 5 percentage points higher than the next best trusted baseline, DIN-SQL. This improvement is primarily because MaskSQL leverages LLMs in a privacy-preserving manner. However, there remains a $\sim$20 point gap between MaskSQL and the best-performing untrusted baseline. We also observe in the experiments that the direct prompting GPT-4.1 outperforms

基于执行的生成查询评估 [26]。为了展示在更复杂查询上最先进的基于大语言模型（LLM）和小语言模型（SLM）的文本到 SQL 模型之间的差距，从 BIRD 开发集拆分中选择了一个具有挑战性的 300 条目子集。特别地，我们选择了涉及复杂 SQL 模式的示例，例如嵌套查询、集合操作（如 INTERSECT）以及多重连接。为了生成用于隐私测量的真实数据，使用 GPT-4.1 [37] 注释应被抽象化的令牌，然后进行人工审查和更正。对于 MaskSQL，Qwen-2.5-7B-Instruct [46] 作为可信的本地 SLM，而 GPT-4.1 作为不可信的 LLM，用于从抽象化提示生成 SQL 查询。

我们在两种隐私政策下评估 MaskSQL。

- Ψ_F：完整的隐私政策，如第4节所定义，其中整个数据库架构及相关值被视为敏感信息。
- Ψ_C：一种基于类别的策略，其中只有与人名、职业和地点概念相关的标记被抽象化。正式定义见附录A。

6.1 基线

我们将 MaskSQL 与几个基准方法进行比较，包括三种最先进的文本到 SQL 框架和直接的语言模型提示。直接提示基准采用简单的少量示例提示，让语言模型根据给定的自然语言问题和数据库模式生成 SQL 查询。DAIL-SQL [13] 在撰写本文时在 Spider 基准 [54] 上的开源解决方案中排名第一，它在少量示例提示的基础上，采用额外策略以提高 SQL 生成的准确性。DIN-SQL [38] 在开源框架中排名第二，它将文本到 SQL 的转换分解为更小的任务以提高准确性。对于直接提示、DAIL-SQL 和 DIN-SQL，我们考虑了可信设置，其中使用 Qwen2.5-7B-Instruct 作为基础语言模型，以及不可信设置，其中使用 GPT-4.1。最后，MSc-SQL [14] 专门为仅使用 SLM 的设置设计。它从不同模型中采样几个候选 SQL 查询，然后选择最佳的一个。

6.2 指标

我们对每个基线在实用性、效率和隐私指标上进行评估。对于实用性评估，我们使用执行准确率，如 BIRD 基准中定义的那样。效率通过每次查询生成的平均令牌使用量来衡量。对于隐私，我们定义了两个指标。

屏蔽召回率 (MR)：该指标定义为正确抽象的标记数量与自然语言问题中真实敏感标记总数的比率 $\mathcal{Q}$。数值越高表示对敏感标记的保护越多，从而隐私性更好。

重新识别得分 (RI)：该指标衡量 NL 问题 $\mathcal{Q}'$ 中被抽象化的 token 无法被对手重新识别的比例。具体来说，我们使用抽象化的 NL 问题 $\mathcal{Q}'$ 和模式 $\mathcal{S}'$ 提示 GPT-4.1，并指示其推断原始 token。然后，该得分计算为 LLM 无法恢复的 token 数量与 $\mathcal{Q}'$ 中抽象化 token 总数的比值。

这些指标的正式定义在附录中提供 B.

6.3 结果

准确性。表1显示了不同基线的执行准确率。基于不可信LLM的方法表现明显优于基于可信SLM的方法，这在预期之中，因为LLM具有额外的能力。然而，直接使用LLM会将敏感数据暴露于隐私风险之中。MaskSQL的表现优于所有仅使用可信SLM的方法，在完全隐私策略下实现了55.66%的执行准确率，Ψ_F，比次佳的可信基线DIN-SQL高出5个百分点。这一提升主要是因为MaskSQL以隐私保护的方式利用了LLM。然而，MaskSQL与表现最好的不可信基线之间仍存在∼20个百分点的差距。我们还在实验中观察到，直接提示GPT-4.1的表现优于

Framework	Execution Accuracy	Token Usage	Trusted
Direct Prompting + Qwen-2.5-7B-Instruct	34.33%	1,380	Yes
DAIL-SQL + Qwen-2.5-7B-Instruct	44.33%	3,492	Yes
Fine-Tuned MSc-SQL	48.33%	8,342	Yes
DIN-SQL + Qwen-2.5-7B-Instruct	50.66%	24,812	Yes
MaskSQL (Ψ_F)	**55.66%**	**6,114**	**Yes**
MaskSQL (Ψ_C)	**62.66%**	**6,757**	**Yes**
DAIL-SQL + GPT-4.1	63.33%	3,385	No
DIN-SQL + GPT-4.1	73.66%	23,036	No
Direct Prompting + GPT-4.1	75.66%	1,352	No

Table 1: Execution accuracy and token usage of MaskSQL compared to other text-to-SQL frameworks on a subset of the BIRD development set. MaskSQL outperforms SLM-based frameworks and achieves better token efficiency than MSc-SQL and DIN-SQL. By adopting a more permissive privacy policy, MaskSQL (Ψ_C) improves execution accuracy while still preserving privacy. Pure LLM-based methods have the best accuracy, but do not protect sensitive data.

both DIN-SQL and DAIL-SQL. This is a surprising result, as these methods are supposed to enhance simple LLM prompting and yield higher accuracy. We discuss this further in Appendix D.

Efficiency. Table 1 also reports the average token usage of each framework per SQL generation. For frameworks that make more than one LM call, we compute the total token usage across all calls. As shown in the table, direct prompting with Qwen2.5 and GPT-4.1 exhibits the lowest token usage, followed by DAIL-SQL and MSc-SQL, while DIN-SQL consumes the most. MaskSQL uses a competitive number of tokens compared to SLM-based frameworks, especially when considering execution accuracy gains. For instance, it requires a quarter of the tokens consumed by the next best trusted model. The additional token usage in MaskSQL primarily comes from the intermediate linking steps and error corrections.

Privacy. Figure 2 presents the privacy scores of MaskSQL measured by the masking recall and re-identification score metrics. In Figure 2, *ground-truth masking* refers to abstracting all ground-truth tokens, providing an upper bound for masking recall and re-identification scores. Under the full policy, Ψ_F, MaskSQL achieves a masking recall of 61.36%, demonstrating that a large portion of sensitive tokens are effectively abstracted. The re-identification score measures robustness against adversarial recovery of the abstract tokens. With GPT-4.1 acting as the attacker, MaskSQL achieves a re-identification score of 75.47%, indicating that three-quarters of abstracted tokens could not be inferred from context. With ground-truth masking, this score is $\sim$86%. The relatively small gap highlights that MaskSQL is robust to contextual information leakage despite not abstracting all tokens.

We further analyze the impact of using a more permissive privacy policy. As shown in Table 1, using the category-based policy, Ψ_C, increases execution accuracy by 7 points compared to Ψ_F at the cost of a 27 point drop in masking recall, which is expected, since fewer tokens are abstracted when only tokens related to specific concepts (name, location, and occupation) are considered. This highlights the flexibility of policy-based abstraction in trading off privacy for higher accuracy, depending on the user's needs. Note that for MaskSQL under the Ψ_C setup, masking recall is computed as the ratio of correctly abstracted tokens in Ψ_C to all ground-truth sensitive tokens, rather than restricting the ground-truth tokens to only those included in the policy. Additionally, the re-identification score decreases by only 4 points, indicating that while fewer tokens are abstracted overall, the remaining abstracted tokens remain difficult for the adversary to recover.

7 Conclusion

In this work, we introduced MaskSQL, a framework for text-to-SQL translation that employs an abstraction-based privacy mechanism to preserve sensitive information according to user-defined policies when interacting with LLMs. Rather than relying solely on either SLMs or LLMs, MaskSQL follows a hybrid design that combines private local processing with SLMs and leverages the reasoning

Framework	Execution Accuracy	Token Usage	Trusted
Direct Prompting + Qwen-2.5-7B-Instruct	34.33%	1,380	Yes
DAIL-SQL + Qwen-2.5-7B-Instruct	44.33%	3,492	Yes
Fine-Tuned MSc-SQL	48.33%	8,342	Yes
DIN-SQL + Qwen-2.5-7B-Instruct	50.66%	24,812	Yes
MaskSQL (Ψ_F)	**55.66%**	**6,114**	**Yes**
MaskSQL (Ψ_C)	**62.66%**	**6,757**	**Yes**
DAIL-SQL + GPT-4.1	63.33%	3,385	No
DIN-SQL + GPT-4.1	73.66%	23,036	No
Direct Prompting + GPT-4.1	75.66%	1,352	No

表 1：MaskSQL 在 BIRD 开发集子集上与其他文本到 SQL 框架的执行准确性和令牌使用情况比较。MaskSQL 的表现优于基于 SLM 的框架，并且比 MSc-SQL 和 DIN-SQL 拥有更高的令牌效率。通过采用更宽松的隐私政策，MaskSQL（Ψ_C）在保持隐私的同时提高了执行准确性。纯 LLM 方法的准确性最佳，但不保护敏感数据。

DIN-SQL 和 DAIL-SQL 都是如此。这是一个令人惊讶的结果，因为这些方法本应增强简单的 LLM 提示并产生更高的准确性。我们在附录 D 中对此进行了进一步讨论。

效率。表 1 还报告了每个框架在 SQL 生成过程中平均使用的令牌数。对于进行多次 LM 调用的框架，我们计算所有调用的令牌总使用量。如表中所示，使用 Qwen2.5 和 GPT-4.1 的直接提示展示了最低的令牌使用量，其次是 DAIL-SQL 和 MSc-SQL，而 DIN-SQL 的消耗最高。与基于 SLM 的框架相比，MaskSQL 使用的令牌数具有竞争力，尤其是在考虑执行准确性提升的情况下。例如，它仅需要下一个最佳可信模型使用令牌的一季度。MaskSQL 中额外的令牌使用主要来自中间链接步骤和错误修正。

隐私。图2展示了 MaskSQL 的隐私评分，这些评分是通过掩码召回率和重识别评分指标测量的。在图2中，*ground-truth masking* 指的是抽象化所有真实标记，为掩码召回率和重识别评分提供上限。在完整策略下，Ψ_F，MaskSQL 实现了 61.36% 的掩码召回率，表明大部分敏感标记得到了有效抽象。重识别评分衡量对抽象标记的对抗性恢复的鲁棒性。在 GPT-4.1 作为攻击者的情况下，MaskSQL 实现了 75.47% 的重识别评分，这表明有四分之三的抽象标记无法从上下文推测出。在真实标记掩码情况下，该评分为 $\sim$ 86%。这一相对较小的差距表明，尽管没有抽象所有标记，MaskSQL 对上下文信息泄露仍具有鲁棒性。

我们进一步分析了使用更宽松的隐私策略的影响。如表 1 所示，使用基于类别的策略 Ψ_C 相较于 Ψ_F 将执行准确率提高了 7 个百分点，但掩码召回率下降了 27 个百分点，这是预期之中的，因为当仅考虑与特定概念（姓名、位置和职业）相关的标记时，被抽象的标记更少。这突显了基于策略的抽象在根据用户需求在隐私和更高准确率之间进行权衡的灵活性。请注意，对于 Ψ_C 设置下的 MaskSQL，掩码召回率是计算 Ψ_C 中正确抽象的标记占所有真实敏感标记的比例，而不是仅将真实标记限制在策略包含的标记中。此外，重新识别得分仅下降了 4 个百分点，这表明尽管总体上抽象的标记更少，但剩余的抽象标记仍然难以被对手恢复。

7 结论

在这项工作中，我们介绍了 MaskSQL，一种用于文本到 SQL 翻译的框架，该框架采用基于抽象的隐私机制，根据用户定义的策略在与大型语言模型（LLM）交互时保护敏感信息。MaskSQL 并不是单纯依赖 SLM 或 LLM，而是采用混合设计，将本地私有处理与 SLM 相结合，并利用推理能力

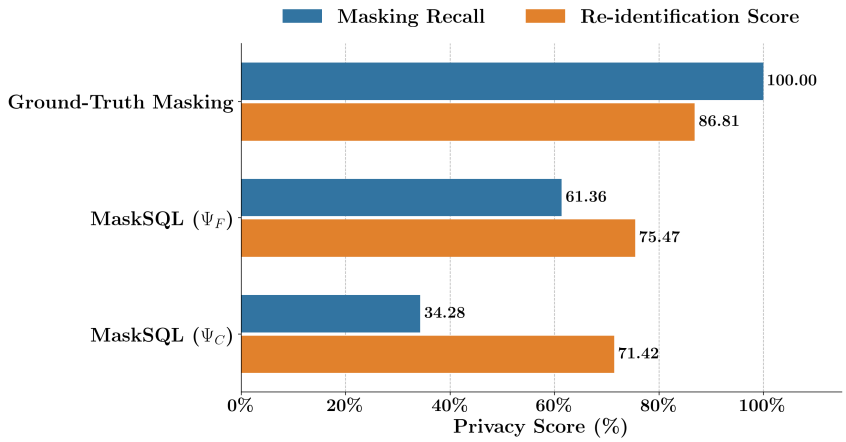


Figure 2: Privacy metrics of MaskSQL compared to ground-truth masking. Higher values indicate stronger privacy preservation.

ability of LLMs. By supporting flexible privacy policies, MaskSQL also enables a privacy-utility trade-off that allows users to tailor abstraction according to their needs. The experiments show that MaskSQL outperforms state-of-the-art SLM-based methods in terms of execution accuracy. Additionally, we implemented a category-based privacy policy that considers only sensitive tokens associated with specific semantic categories to be abstracted. The experimental results demonstrate that this more lenient policy achieves higher execution accuracy than the full privacy policy, highlighting the potential of controlled trade-offs between privacy and utility.

There are several promising directions for future work to be explored. First, since only a subset of tokens is abstracted within the NL question, the surrounding context may be exploited to re-identify the abstract tokens. Currently, we quantify this privacy risk by measuring the re-identification score. However, additional privacy measures can be applied to mitigate such potential leakage. Next, while MaskSQL uses an LLM backbone for SQL generation, its accuracy still falls behind other LLM-based frameworks. Future work will explore fine-tuning SLMs, improved schema linking, and token-efficient strategies to reduce this gap. Further, we plan to consider integrating provable privacy mechanisms, such as differential privacy, into the MaskSQL pipeline. In addition to the privacy measurements leveraged in this work, provable guarantees enhance the user's trust in the system. Finally, the abstraction-based approach in MaskSQL is not limited to the text-to-SQL translation task. We envision the abstraction approach to be applicable to tasks such as code generation, debugging, and data analysis, where reference and relational information are sufficient to perform the task. Characterizing the class of tasks to which this approach applies will be the subject of future work.

References

[1] Mistral AI. Mistral-7B-Instruct-v0.2 model card. `https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.2`, 2023.

[2] Yoshimasa Akimoto, Kazuto Fukuchi, Youhei Akimoto, and Jun Sakuma. Privformer: Privacy-preserving transformer with MPC. In *8th European Symposium on Security and Privacy*, pages 392–410. IEEE, 2023.

[3] Christopher Baik, Zhongjun Jin, Michael Cafarella, and H. V. Jagadish. Duoquest: A Dual-Specification System for Expressive SQL Queries. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, page 2319–2329, 2020. ISBN 9781450367356. doi: 10.1145/3318464.3389776.

[4] R.J. Bayardo and Rakesh Agrawal. Data privacy through optimal k-anonymization. In *21st International Conference on Data Engineering*, pages 217–228, 2005. doi: 10.1109/ICDE.2005.42.

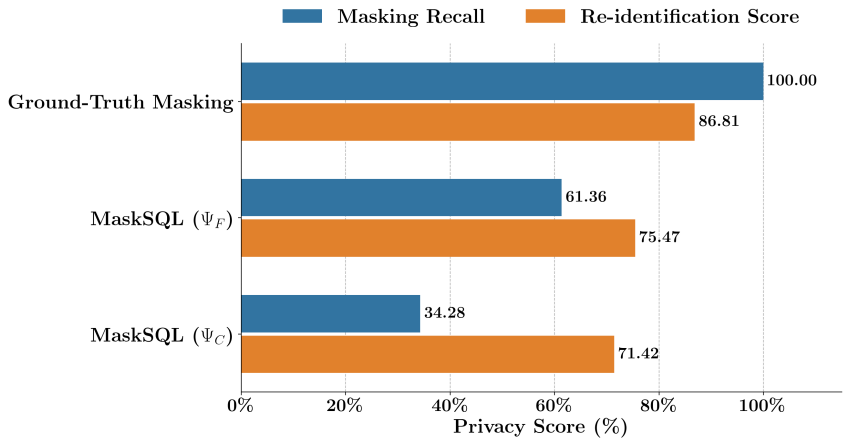


图2：MaskSQL 与真实掩码的隐私指标比较。数值越高表示隐私保护越强。

大型语言模型（LLMs）的能力。通过支持灵活的隐私策略，MaskSQL 还实现了隐私与实用性的权衡，使用户能够根据自身需求定制抽象。实验显示，MaskSQL 在执行准确性方面优于基于最先进 SLM 的方法。此外，我们还实现了一种基于类别的隐私策略，该策略仅考虑与特定语义类别相关的敏感标记进行抽象。实验结果表明，这种较宽松的策略在执行准确性上高于完整隐私策略，凸显了在隐私与实用性之间进行可控权衡的潜力。

未来有几个有前景的研究方向值得探索。首先，由于在自然语言问题中只有一部分标记被抽象化，因此可以利用周围的上下文来重新识别抽象标记。目前，我们通过测量重新识别得分来量化这种隐私风险。然而，可以应用额外的隐私措施来减轻这种潜在的泄露风险。接下来，虽然 MaskSQL 使用 LLM 作为 SQL 生成的骨干，其准确性仍落后于其他基于 LLM 的框架。未来的工作将探索微调 SLM、改进模式链接以及提高标记利用效率的策略以缩小这一差距。此外，我们计划考虑将可证明的隐私机制，如差分隐私，集成到 MaskSQL 流程中。除了本研究中使用的隐私测量外，可证明的保障能够增强用户对系统的信任。最后，MaskSQL 的基于抽象的方法并不限于文本到 SQL 的翻译任务。我们设想

参考文献

[1] Mistral AI. Mistral-7B-Instruct-v0.2 模型卡。https://huggingface.co/mistralai/ Mistral-7B-Instruct-v0.2, 2023。

[2] 秋元 義政, 福地 一翔, 秋元 洋平, 佐久間 淳. Privformer: 基于 MPC 的隐私保护 Transformer. 收录于 *8th European Symposium on Security and Privacy*, 页码 392–410. IEEE, 2023.

[3] Christopher Baik, Zhongjun Jin, Michael Cafarella 和 H. V. Jagadish. Duoquest：一种用于表达性 SQL 查询的双规范系统。收录于 *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, 页码 2319–2329, 2020. ISBN 9781450367356. doi: 10.1145/3318464.3389776.

[4] R.J. Bayardo 和 Rakesh Agrawal. 通过最优 k-匿名化实现数据隐私保护. 载于 *21st International Conference on Data Engineering*, 页 217–228, 2005. doi: 10.1109/ICDE.2005.42.

[5] Antorweep Chakravorty, Chunming Rong, K.R. Jayaram, and Shu Tao. Scalable, Efficient Anonymization with INCOGNITO - Framework & Algorithm. In *2017 IEEE International Congress on Big Data*, 2017. doi: 10.1109/BigDataCongress.2017.15.

[6] Tianyu Chen, Hangbo Bao, Shaohan Huang, Li Dong, Binxing Jiao, Daxin Jiang, Haoyi Zhou, Jianxin Li, and Furu Wei. THE-X: Privacy-Preserving Transformer Inference with Homomorphic Encryption. In *Findings of the Association for Computational Linguistics: ACL 2022*, 2022. doi: 10.18653/v1/2022.findings-acl.277.

[7] Yu Chen, Tingxin Li, Huiming Liu, and Yang Yu. Hide and seek (HaS): A lightweight framework for prompt privacy protection. *arXiv:2309.03057*, 2023.

[8] Amrita Roy Chowdhury, David Glukhov, Divyam Anshumaan, Prasad Chalasani, Nicolas Papernot, Somesh Jha, and Mihir Bellare. Pr$\varepsilon\varepsilon$mpt: Sanitizing Sensitive Prompts for LLMs. *arXiv:2504.05147*, 2025.

[9] Soumi Das, Camila Kolling, Mohammad Aflah Khan, Mahsa Amani, Bishwamittra Ghosh, Qinyuan Wu, Till Speicher, and Krishna P Gummadi. Revisiting Privacy, Utility, and Efficiency Trade-offs when Fine-Tuning Large Language Models. *arXiv:2502.13313*, 2025.

[10] Naihao Deng, Yulong Chen, and Yue Zhang. Recent Advances in Text-to-SQL: A Survey of What We Have and What We Expect. In *Proceedings of the 29th International Conference on Computational Linguistics*, pages 2166–2187, 2022.

[11] Minxin Du, Xiang Yue, Sherman S. M. Chow, Tianhao Wang, Chenyu Huang, and Huan Sun. DP-Forward: Fine-tuning and Inference on Language Models with Differential Privacy in Forward Pass. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, page 2665–2679, 2023. ISBN 9798400700507. doi: 10.1145/3576915.3616592.

[12] Yao Fu, Hao Peng, Litu Ou, Ashish Sabharwal, and Tushar Khot. Specializing smaller language models towards multi-step reasoning. In *International Conference on Machine Learning*, pages 10421–10430. PMLR, 2023.

[13] Dawei Gao, Haibin Wang, Yaliang Li, Xiuyu Sun, Yichen Qian, Bolin Ding, and Jingren Zhou. Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation. *Proc. VLDB Endow.*, 17(5):1132–1145, 2024. ISSN 2150-8097. doi: 10.14778/3641204.3641221.

[14] Satya Krishna Gorti, Ilan Gofman, Zhaoyan Liu, Jiapeng Wu, Noël Vouitsis, Guangwei Yu, Jesse C. Cresswell, and Rasa Hosseinzadeh. MSc-SQL: Multi-sample critiquing small language models for text-to-SQL translation. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 2145–2160, 2025. ISBN 979-8-89176-189-6. doi: 10.18653/v1/2025.naacl-long.107.

[15] Aaron Grattafiori et al. The Llama 3 Herd of Models. *arXiv:2407.21783*, 2024.

[16] Alex Graves and Jürgen Schmidhuber. Framewise phoneme classification with bidirectional lstm and other neural network architectures. *Neural Networks*, 18(5):602–610, 2005. ISSN 0893-6080. doi: https://doi.org/10.1016/j.neunet.2005.06.042.

[17] Charlie Hou, Akshat Shrivastava, Hongyuan Zhan, Rylan Conway, Trang Le, Adithya Sagar, Giulia Fanti, and Daniel Lazar. PrE-Text: Training Language Models on Private Federated Data in the Age of LLMs. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pages 19043–19061, 2024.

[18] Yiming Huang, Jiyu Guo, Wenxin Mao, Cuiyun Gao, Peiyi Han, Chuanyi Liu, and Qing Ling. Exploring the landscape of text-to-sql with large language models: Progresses, challenges and opportunities. *arXiv:2505.23838*, 2025.

[19] Timour Igamberdiev and Ivan Habernal. DP-BART for privatized text rewriting under local differential privacy. In *Findings of the Association for Computational Linguistics: ACL 2023*, pages 13914–13934, 2023. doi: 10.18653/v1/2023.findings-acl.874.

[5] Antorweep Chakravorty, Chunming Rong, K.R. Jayaram, 和 Shu Tao. 使用 INCOGNITO 的可扩展、高效匿名化 - 框架与算法. 收录于 *2017 IEEE International Congress on Big Data*, 2017. doi: 10.1109/BigDataCongress.2017.15.

[6] 陈天宇, 鲍航博, 黄少汉, 董立, 焦宾兴, 蒋大新, 周浩毅, 李建新, 魏夫如. THE-X: 基于同态加密的隐私保护 Transformer 推理. 载于 *Findings of the Association for Computational Linguistics: ACL 2022*, 2022. doi: 10.18653/v1/2022.findings-acl.277.

[7] 陈宇, 李亭新, 刘辉明, 余杨. 捉迷藏（HaS）：一个轻量级的提示隐私保护框架. *arXiv:2309.03057*, 2023.

[8] Amrita Roy Chowdhury, David Glukhov, Divyam Anshumaan, Prasad Chalasani, Nicolas Papernot, Somesh Jha, 和 Mihir Bellare. Prεεmpt: 为大型语言模型清理敏感提示. *arXiv:2504.05147*, 2025.

[9] Soumi Das, Camila Kolling, Mohammad Aflah Khan, Mahsa Amani, Bishwamittra Ghosh, Qinyuan Wu, Till Speicher, 和 Krishna P Gummadi. 在微调大型语言模型时重新审视隐私、实用性和效率的权衡。*arXiv:2502.13313*, 2025.

[10] 邓乃豪, 陈玉龙, 张越. 文本到SQL的最新进展：我们已有的与我们期望的综述. 载于 *Proceedings of the 29th International Conference on Computational Linguistics*, 页码 2166–2187, 2022.

[11] 杜敏鑫, 岳翔, 周思铭, 王天昊, 黄晨宇, 孙欢. DP-Forward: 在前向传播中使用差分隐私对语言模型进行微调和推理. 载于 *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, 第2665–2679页, 2023年. ISBN 9798400700507. doi: 10.1145/ 3576915.3616592.

[12] 姚福, 彭浩, 欧力图, 阿希什·萨巴瓦尔, 和 图沙尔·科特. 将较小的语言模型专门用于多步骤推理. 载于 *International Conference on Machine Learning*, 第10421–10430页. PMLR, 2023.

[13] 高大伟, 王海滨, 李亚亮, 孙秀玉, 钱伊晨, 丁博林, 周景仁. 基于大语言模型的文本到SQL: 基准评估. *Proc. VLDB Endow.*, 17(5):1132–1145, 2024. ISSN 2150-8097. doi: 10.14778/3641204.3641221.

[14] Satya Krishna Gorti, Ilan Gofman, Zhaoyan Liu, Jiapeng Wu, Noël Vouitsis, Guangwei Yu, Jesse C. Cresswell 和 Rasa Hosseinzadeh. MSc-SQL：用于文本到 SQL 翻译的多样本批评小型语言模型。发表于 *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, 页 2145–2160, 2025。ISBN 979-8-89176-189-6。doi: 10.18653/v1/2025.naacl-long.107.

[15] Aaron Grattafiori 等。《Llama 3 模型群》。*arXiv:2407.21783*, 2024。

[16] Alex Graves 和 Jürgen Schmidhuber。使用双向 LSTM 及其他神经网络架构进行逐帧音素分类。*Neural Networks*, 18(5):602–610, 2005。ISSN 0893-6080。doi: https://doi.org/10.1016/j.neunet.2005.06.042.

[17] Charlie Hou, Akshat Shrivastava, Hongyuan Zhan, Rylan Conway, Trang Le, Adithya Sagar, Giulia Fanti, 和 Daniel Lazar. PrE-Text：在大语言模型时代在私有联邦数据上训练语言模型。发表于 *Proceedings of the 41st International Conference on Machine Learning*, 第 235 卷, 页 19043–19061, 2024 年。

[18] 黄一鸣, 郭继宇, 毛文新, 高翠云, 韩佩怡, 刘传义, 以及凌青. 利用大型语言模型探索文本到SQL的研究现状：进展、挑战与机遇. *arXiv:2505.23838*, 2025.

[19] Timour Igamberdiev 和 Ivan Habernal. 在局部差分隐私下用于文本私有化重写的 DP-BART. 见 *Findings of the Association for Computational Linguistics: ACL 2023*, 第 13914–13934 页, 2023. doi: 10.18653/v1/2023.findings-acl.874.

[20] Zhigang Kan, Linbo Qiao, Hao Yu, Liwen Peng, Yifu Gao, and Dongsheng Li. Protecting user privacy in remote conversational systems: A privacy-preserving framework based on text sanitization. *arXiv:2306.08223*, 2023.

[21] Đorđe Klisura and Anthony Rios. Unmasking Database Vulnerabilities: Zero-Knowledge Schema Inference Attacks in Text-to-SQL Systems. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 6954–6976, 2025. ISBN 979-8-89176-195-7. doi: 10.18653/v1/2025.findings-naacl.386.

[22] Fangyu Lei, Jixuan Chen, Yuxiao Ye, Ruisheng Cao, Dongchan Shin, Hongjin SU, Zhaoqing Suo, Hongcheng Gao, Wenjing Hu, Pengcheng Yin, Victor Zhong, Caiming Xiong, Ruoxi Sun, Qian Liu, Sida Wang, and Tao Yu. Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows. In *The Thirteenth International Conference on Learning Representations*, 2025.

[23] Wenqiang Lei, Weixin Wang, Zhixin Ma, Tian Gan, Wei Lu, Min-Yen Kan, and Tat-Seng Chua. Re-examining the Role of Schema Linking in Text-to-SQL. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6943–6954, 2020. doi: 10.18653/v1/2020.emnlp-main.564.

[24] Haoyang Li, Jing Zhang, Cuiping Li, and Hong Chen. RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(11):13067–13075, 2023. doi: 10.1609/aaai.v37i11.26535.

[25] Haoyang Li, Jing Zhang, Hanbing Liu, Ju Fan, Xiaokang Zhang, Jun Zhu, Renjie Wei, Hongyan Pan, Cuiping Li, and Hong Chen. CodeS: Towards Building Open-source Language Models for Text-to-SQL. *Proc. ACM Manag. Data*, 2(3), 2024. doi: 10.1145/3654930.

[26] Jinyang Li, Binyuan Hui, Ge Qu, Jiaxi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, Xuanhe Zhou, Ma Chenhao, Guoliang Li, Kevin Chang, Fei Huang, Reynold Cheng, and Yongbin Li. Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs. In *Advances in Neural Information Processing Systems*, volume 36, pages 42330–42357, 2023.

[27] Siyan Li, Vethavikashini Chithrra Raghuram, Omar Khattab, Julia Hirschberg, and Zhou Yu. PAPILLON: Privacy Preservation from Internet-based and Local Language Model Ensembles. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 3371–3390, 2025. ISBN 979-8-89176-189-6. doi: 10.18653/v1/2025.naacl-long.173.

[28] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv:1907.11692*, 2019.

[29] Natasha Lomas. Italy orders ChatGPT blocked citing data protection concerns. *TechCrunch*, March 2023. URL `https://techcrunch.com/2023/03/31/chatgpt-blocked-italy/`.

[30] Jianqiang Ma, Zeyu Yan, Shuai Pang, Yang Zhang, and Jianping Shen. Mention Extraction and Linking for SQL Query Generation. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 6936–6942, 2020. doi: 10.18653/v1/2020.emnlp-main.563.

[31] Karime Maamari, Fadhil Abubaker, Daniel Jaroslawicz, and Amine Mhedhbi. The Death of Schema Linking? Text-to-SQL in the Age of Well-Reasoned Language Models. *arXiv:2408.07702*, 2024.

[32] Peihua Mai, Ran Yan, Zhe Huang, Youjia Yang, and Yan Pang. Split-and-Denoise: Protect large language model inference with local differential privacy. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pages 34281–34302, 2024.

[33] Jimit Majmudar, Christophe Dupuy, Charith Peris, Sami Smaili, Rahul Gupta, and Richard Zemel. Differentially private decoding in large language models. *arXiv:2205.13621*, 2022.

[20] 阚志刚, 乔林波, 余浩, 彭立文, 高一夫, 李东升. 在远程对话系统中保护用户隐私：基于文本清理的隐私保护框架. *arXiv:2306.08223*, 2023.

[21] Đor đe Klisura 和 Anthony Rios. 揭示数据库漏洞：文本到 SQL 系统中的零知识模式推断攻击. 收录于 *Findings of the Association for Compu- tational Linguistics: NAACL 2025*, 第 6954–6976 页, 2025. ISBN 979-8-89176-195-7. doi: 10.18653/v1/2025.findings-naacl.386.

[22] 雷方宇, 陈季旋, 叶宇霄, 曹瑞生, 申东灿, 苏宏进, 索兆清, 高鸿成, 胡文静, 尹鹏程, 赵维克, 熊采明, 孙若曦, 刘谦, 王思达, 余涛. Spider 2.0: 在真实企业文本到SQL工作流中评估语言模型. 发表在 *The Thirteenth International Conference on Learning Representations*, 2025.

[23] 雷文强, 王伟心, 马志新, 甘田, 卢伟, 简民彦, 和蔡达生. 重新审视模式链接在文本到SQL中的作用. 载于 *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 第6943–6954页, 2020. doi: 10.18653/v1/2020.emnlp-main.564.

[24] 李浩阳, 张晶, 李翠平, 陈宏. RESDSQL: 分离模式链接与骨架解析的文本到SQL方法. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(11):13067–13075, 2023. doi: 10.1609/aaai.v37i11.26535.

[25] 李浩洋, 张晶, 刘涵冰, 樊菊, 张晓康, 朱俊, 韦人杰, 潘红燕, 李翠萍, 陈宏. CodeS: 构建开源文本到SQL语言模型的探索. *Proc. ACM Manag. Data*, 2(3), 2024. doi: 10.1145/3654930.

[26] 李金阳, 惠滨源, 曲戈, 杨佳溪, 李彬华, 李博文, 王柏麟, 秦博文, 耿锐英, 霍楠, 周宣和, 陈浩马, 李国梁, Kevin Chang, 黄飞, 郑瑞诺, 李永斌. 大型语言模型能否已作为数据库接口使用？面向大规模数据库的文本到SQL的BIg Bench. 发表于 *Advances in Neural Information Processing Systems*, 第36卷, 页42330–42357, 2023年.

[27] Siyan Li, Vethavikashini Chithrra Raghuram, Omar Khattab, Julia Hirschberg, 和 Zhou Yu. PAPILLON：基于互联网和本地语言模型集成的隐私保护。在 *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*，第 3371–3390 页，2025 年。ISBN 979-8-89176-189-6。doi: 10.18653/v1/2025.naacl-long.173.

[28] 刘银汉, Myle Ott, Naman Goyal, 杜静飞, Mandar Joshi, 陈丹奇, Omer Levy, Mike Lewis, Luke Zettlemoyer, 和 Veselin Stoyanov. RoBERTa：一种鲁棒优化的 BERT 预训练方法. *arXiv:1907.11692*, 2019.

[29] Natasha Lomas。意大利因数据保护问题下令封锁 ChatGPT。*TechCrunch*，2023 年 3 月。网址 https://techcrunch.com/2023/03/31/chatgpt-blocked-italy/。

[30] 马建强, 颜泽宇, 庞帅, 张杨, 沈建平. SQL 查询生成的提及提取与链接. 载于 *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 页码 6936–6942, 2020. doi: 10.18653/v1/2020.emnlp-main.563.

[31] Karime Maamari, Fadhil Abubaker, Daniel Jaroslawicz, 和 Amine Mhedhbi. 架构链接的终结？在推理能力强的语言模型时代的文本到SQL。*arXiv:2408.07702*, 2024.

[32] 麦培华，闫冉，黄喆，杨有家，庞岩。分割与去噪：使用本地差分隐私保护大型语言模型推理。在 *Proceedings of the 41st International Conference on Machine Learning*，第 235 卷，34281–34302 页，2024 年。

[33] Jimit Majmudar, Christophe Dupuy, Charith Peris, Sami Smaili, Rahul Gupta, 和 Richard Zemel. 大型语言模型中的差分隐私解码. *arXiv:2205.13621*, 2022.

[34] H. Brendan McMahan, Daniel Ramage, Kunal Talwar, and Li Zhang. Learning Differentially Private Recurrent Language Models. In *International Conference on Learning Representations*, 2018.

[35] McMahon, Liv. Hundreds of thousands of Grok chats exposed in Google results. *BBC News*, August 2025. URL `https://www.bbc.com/news/articles/cdrkmk00jy0o`.

[36] Shervin Minaee, Nal Kalchbrenner, Erik Cambria, Narjes Nikzad, Meysam Chenaghlu, and Jianfeng Gao. Deep Learning–based Text Classification: A Comprehensive Review. *ACM Comput. Surv.*, 54(3), 2021. ISSN 0360-0300. doi: 10.1145/3439726.

[37] OpenAI. GPT-4.1. `https://openai.com/index/gpt-4-1/`, 2025.

[38] Mohammadreza Pourreza and Davood Rafiei. DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction. In *Advances in Neural Information Processing Systems*, volume 36, pages 36339–36348, 2023.

[39] Mohammadreza Pourreza and Davood Rafiei. DTS-SQL: Decomposed Text-to-SQL with Small Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 8212–8220, 2024. doi: 10.18653/v1/2024.findings-emnlp.481.

[40] Bowen Qin, Binyuan Hui, Lihan Wang, Min Yang, Jinyang Li, Binhua Li, Ruiying Geng, Rongyu Cao, Jian Sun, Luo Si, et al. A Survey on Text-to-SQL Parsing: Concepts, Methods, and Future Directions. *arXiv:2208.13629*, 2022.

[41] Abdul Quamar, Vasilis Efthymiou, Chuan Lei, and Fatma Özcan. Natural language interfaces to data. *Foundations and Trends® in Databases*, 11(4):319–414, 2022. ISSN 1931-7883. doi: 10.1561/1900000078.

[42] Pierangela Samarati and Latanya Sweeney. Protecting Privacy when Disclosing Information: k-anonymity and Its Enforcement through Generalization and Suppression. 1998.

[43] Lei Sheng and Shuai-Shuai Xu. SLM-SQL: An Exploration of Small Language Models for Text-to-SQL. *arXiv:2507.22478*, 2025.

[44] Liang Shi, Zhengju Tang, Nan Zhang, Xiaotong Zhang, and Zhi Yang. A Survey on Employing Large Language Models for Text-to-SQL Tasks. *ACM Comput. Surv.*, 2025. ISSN 0360-0300. doi: 10.1145/3737873.

[45] Gemma Team, Morgane Riviere, Shreya Pathak, Pier Giuseppe Sessa, Cassidy Hardin, Surya Bhupatiraju, Léonard Hussenot, Thomas Mesnard, Bobak Shahriari, Alexandre Ramé, Johan Ferret, Peter Liu, Pouya Tafti, Abe Friesen, Michelle Casbon, Sabela Ramos, Ravin Kumar, Charline Le Lan, Sammy Jerome, Anton Tsitsulin, Nino Vieillard, Piotr Stanczyk, Sertan Girgin, Nikola Momchev, Matt Hoffman, Shantanu Thakoor, Jean-Bastien Grill, Behnam Neyshabur, Olivier Bachem, Alanna Walton, Aliaksei Severyn, Alicia Parrish, Aliya Ahmad, Allen Hutchison, Alvin Abdagic, Amanda Carl, Amy Shen, Andy Brock, Andy Coenen, Anthony Laforge, Antonia Paterson, Ben Bastian, Bilal Piot, Bo Wu, Brandon Royal, Charlie Chen, Chintu Kumar, Chris Perry, Chris Welty, Christopher A. Choquette-Choo, Danila Sinopalnikov, David Weinberger, Dimple Vijaykumar, Dominika Rogozińska, Dustin Herbison, Elisa Bandy, Emma Wang, Eric Noland, Erica Moreira, Evan Senter, Evgenii Eltyshev, Francesco Visin, Gabriel Rasskin, Gary Wei, Glenn Cameron, Gus Martins, Hadi Hashemi, Hanna Klimczak-Plucińska, Harleen Batra, Harsh Dhand, Ivan Nardini, Jacinda Mein, Jack Zhou, James Svensson, Jeff Stanway, Jetha Chan, Jin Peng Zhou, Joana Carrasqueira, Joana Iljazi, Jocelyn Becker, Joe Fernandez, Joost van Amersfoort, Josh Gordon, Josh Lipschultz, Josh Newlan, Ju yeong Ji, Kareem Mohamed, Kartikeya Badola, Kat Black, Katie Millican, Keelin McDonell, Kelvin Nguyen, Kiranbir Sodhia, Kish Greene, Lars Lowe Sjoesund, Lauren Usui, Laurent Sifre, Lena Heuermann, Leticia Lago, Lilly McNealus, Livio Baldini Soares, Logan Kilpatrick, Lucas Dixon, Luciano Martins, Machel Reid, Manvinder Singh, Mark Iverson, Martin Görner, Mat Velloso, Mateo Wirth, Matt Davidow, Matt Miller, Matthew Rahtz, Matthew Watson, Meg Risdal, Mehran Kazemi, Michael Moynihan, Ming Zhang, Minsuk Kahng, Minwoo Park, Mofi Rahman, Mohit Khatwani, Natalie Dao, Nenshad Bardoliwalla, Nesh Devanathan, Neta Dumai, Nilay Chauhan, Oscar Wahltinez, Pankil Botarda, Parker Barnes, Paul Barham, Paul Michel, Pengchong Jin,

[34] H. Brendan McMahan, Daniel Ramage, Kunal Talwar, 和 Li Zhang. 学习差分隐私递归语言模型。发表于 *International Conference on Learning Representations*, 2018。

[35] 麦克马洪，利夫。数十万条Grok聊天记录在谷歌搜索结果中被曝光。*BBC News*, 2025年8月。网址 https://www.bbc.com/news/articles/cdrkmk00jy0o.

[36] Shervin Minaee, Nal Kalchbrenner, Erik Cambria, Narjes Nikzad, Meysam Chenaghlu 和 Jianfeng Gao。基于深度学习的文本分类：全面综述。*ACM Comput. Surv.*, 54(3), 2021。ISSN 0360-0300。doi: 10.1145/3439726。

[37] OpenAI. GPT-4.1. https://openai.com/index/gpt-4-1/, 2025.

[38] Mohammadreza Pourreza 和 Davood Rafiei. DIN-SQL：基于自我纠正的文本到 SQL 的分解上下文学习。发表于 *Advances in Neural Information Processing Systems*, 第36卷，页36339–36348, 2023年。

[39] Mohammadreza Pourreza 和 Davood Rafiei. DTS-SQL: 使用小型大型语言模型的分解文本到 SQL. 发表在 *Findings of the Association for Computational Linguistics: EMNLP 2024*, 第 8212–8220 页, 2024. doi: 10.18653/v1/2024.findings-emnlp.481.

[40] 秦博文、惠斌元、王立寒、杨敏、李晋阳、李彬华、耿瑞英、曹荣宇、孙坚、斯罗 等. 文本到SQL解析综述：概念、方法与未来方向. *arXiv:2208.13629*, 2022.

[41] Abdul Quamar, Vasilis Efthymiou, Chuan Lei, 和 Fatma Özcan. 数据的自然语言接口。*Foundations and Trends® in Databases*, 11(4):319–414, 2022. ISSN 1931-7883. doi: 10.1561/1900000078.

[42] Pierangela Samarati 和 Latanya Sweeney. 在披露信息时保护隐私：k-匿名性及其通过泛化和抑制的实施. 1998.

[43] 雷声 和 徐帅帅. SLM-SQL：小型语言模型在文本到SQL中的探索. *arXiv:2507.22478*, 2025.

[44] 梁石，郑聚唐，南张，肖桐张，杨志。《大型语言模型在文本到SQL任务中的应用综述》。*ACM Comput. Surv.*，2025年。ISSN 0360-0300。doi: 10.1145/3737873。[45] Gemma团队，Morgane Riviere，Shreya Pathak，Pier Giuseppe Sessa，Cassidy Hardin，Surya Bhupatiraju，Léonard Hussenot，Thomas Mesnard，Bobak Shahriari，Alexandre Ramé，Johan Ferret，Peter Liu，Pouya Tafti，Abe Friesen，Michelle Casbon，Sabela Ramos，Ravin Kumar，Charline Le Lan，Sammy Jerome，Anton Tsitsulin，Nino Vieillard，Piotr Stanczyk，Sertan Girgin，Nikola Momchev，Matt Hoffman，Shantanu Thakoor，Jean-Bastien Grill，Behnam Neyshabur，Olivier Bachem，Alanna Walton，Aliaksei Severyn，Alicia Parrish，Aliya Ahmad，Allen Hutchison，Alvin Abdagic，Amanda Carl，Amy Shen，Andy Brock，Andy Coenen，Anthony Laforge，Antonia Paterson，Ben Bastian，Bilal Piot，Bo Wu，Brandon Royal，Charlie Chen，Chintu Kumar，Chris Perry，Chris Welty，Christopher A. Choquette-Choo，Danila Sinopalnikov，David Weinberge

Petko Georgiev, Phil Culliton, Pradeep Kuppala, Ramona Comanescu, Ramona Merhej, Reena Jana, Reza Ardeshir Rokni, Rishabh Agarwal, Ryan Mullins, Samaneh Saadat, Sara Mc Carthy, Sarah Cogan, Sarah Perrin, Sébastien M. R. Arnold, Sebastian Krause, Shengyang Dai, Shruti Garg, Shruti Sheth, Sue Ronstrom, Susan Chan, Timothy Jordan, Ting Yu, Tom Eccles, Tom Hennigan, Tomas Kocisky, Tulsee Doshi, Vihan Jain, Vikas Yadav, Vilobh Meshram, Vishal Dharmadhikari, Warren Barkley, Wei Wei, Wenming Ye, Woohyun Han, Woosuk Kwon, Xiang Xu, Zhe Shen, Zhitao Gong, Zichuan Wei, Victor Cotruta, Phoebe Kirk, Anand Rao, Minh Giang, Ludovic Peran, Tris Warkentin, Eli Collins, Joelle Barral, Zoubin Ghahramani, Raia Hadsell, D. Sculley, Jeanine Banks, Anca Dragan, Slav Petrov, Oriol Vinyals, Jeff Dean, Demis Hassabis, Koray Kavukcuoglu, Clement Farabet, Elena Buchatskaya, Sebastian Borgeaud, Noah Fiedel, Armand Joulin, Kathleen Kenealy, Robert Dadashi, and Alek Andreev. Gemma 2: Improving Open Language Models at a Practical Size. *arXiv:2408.00118*, 2024.

[46] Qwen Team, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report. *arXiv:2505.09388*, 2025.

[47] Meng Tong, Kejiang Chen, Jie Zhang, Yuang Qi, Weiming Zhang, Nenghai Yu, Tianwei Zhang, and Zhikun Zhang. InferDPT: Privacy-preserving Inference for Black-box Large Language Models. *IEEE Transactions on Dependable and Secure Computing*, pages 1–16, 2025. doi: 10.1109/TDSC.2025.3550389.

[48] Bailin Wang, Richard Shin, Xiaodong Liu, Oleksandr Polozov, and Matthew Richardson. RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 7567–7578, 2020. doi: 10.18653/v1/2020.acl-main.677.

[49] Shang Wang, Tianqing Zhu, Bo Liu, Ming Ding, Xu Guo, Dayong Ye, Wanlei Zhou, and Philip S Yu. Unique security and privacy threats of large language model: A comprehensive survey. *arXiv:2406.07973*, 2024.

[50] Jason Wei, Yi Tay, Rishi Bommasani, Colin Raffel, Barret Zoph, Sebastian Borgeaud, Dani Yogatama, Maarten Bosma, Denny Zhou, Donald Metzler, et al. Emergent abilities of large language models. *arXiv:2206.07682*, 2022.

[51] Xiaojun Xu, Chang Liu, and Dawn Song. Sqlnet: Generating structured queries from natural language without reinforcement learning. *arXiv:1711.04436*, 2017.

[52] Biwei Yan, Kun Li, Minghui Xu, Yueyan Dong, Yue Zhang, Zhaochun Ren, and Xiuzhen Cheng. On protecting the data privacy of Large Language Models (LLMs) and LLM agents: A literature review. *High-Confidence Computing*, 5(2), 2025. ISSN 2667-2952. doi: 10.1016/j.hcc.2025.100300.

[53] Tao Yu, Zifan Li, Zilin Zhang, Rui Zhang, and Dragomir Radev. TypeSQL: Knowledge-Based Type-Aware Neural Text-to-SQL Generation. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pages 588–594, 2018. doi: 10.18653/v1/N18-2093.

[54] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 3911–3921, 2018. doi: 10.18653/v1/D18-1425.

[55] Xiang Yue, Minxin Du, Tianhao Wang, Yaliang Li, Huan Sun, and Sherman S. M. Chow. Differential privacy for text analytics via natural text sanitization. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 3853–3866, 2021. doi: 10.18653/v1/2021.findings-acl.337.

Petko Georgiev、Phil Culliton、Pradeep Kuppala、Ramona Comanescu、Ramona Merhej、Reena Jana、Reza Ardeshir Rokni、Rishabh Agarwal、Ryan Mullins、Samaneh Saadat、Sara Mc Carthy、Sarah Cogan、Sarah Perrin、Sébastien M. R. Arnold、Sebastian Krause、Shengyang Dai、Shruti Garg、Shruti Sheth、Sue Ronstrom、Susan Chan、Timothy Jordan、Ting Yu、Tom Eccles、Tom Hennigan、Tomas Kocisky、Tulsee Doshi、Vihan Jain、Vikas Yadav、Vilobh Meshram、Vishal Dharmadhikari、Warren Barkley、Wei Wei、Wenming Ye、Woohyun Han、Woosuk Kwon、Xiang Xu、Zhe Shen、Zhitao Gong、Zichuan Wei、Victor Cotruta、Phoebe Kirk、Anand Rao、Minh Giang、Ludovic Peran、Tris Warkentin、Eli Collins、Joelle Barral、Zoubin Ghahramani、Raia Hadsell、D. Sculley、Jeanine Banks、Anca Dragan、Slav Petrov、Oriol Vinyals、Jeff Dean、Demis Hassabis、Koray Kavukcuoglu、Clement Farabet、Elena Buchatskaya、Sebastian Borgeaud、Noah Fiedel、Armand Joulin、Kathleen Kenealy、Robert Dadashi 和 Alek Andreev。Gemma 2：改进开放语言模型

[56] Jiangou Zhan, Wenhui Zhang, Zheng Zhang, Huanran Xue, Yao Zhang, and Ye Wu. Portcullis: A Scalable and Verifiable Privacy Gateway for Third-Party LLM Inference. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(1):1022–1030, 2025. doi: 10.1609/aaai.v39i1.32088.

[57] Bin Zhang, Yuxiao Ye, Guoqing Du, Xiaoru Hu, Zhishuai Li, Sun Yang, Chi Harold Liu, Rui Zhao, Ziyue Li, and Hangyu Mao. Benchmarking the Text-to-SQL Capability of Large Language Models: A Comprehensive Evaluation. *arXiv:2403.02951*, 2024.

[58] Victor Zhong, Caiming Xiong, and Richard Socher. Seq2sql: Generating structured queries from natural language using reinforcement learning. *arXiv:1709.00103*, 2017.

[59] Xiaohu Zhu, Qian Li, Lizhen Cui, and Yongkang Liu. Large Language Model Enhanced Text-to-SQL Generation: A Survey. *arXiv:2410.06011*, 2024.

A Category-Based Policy Definition

Let Ψ_F be the full policy under which the entire schema and database values are abstracted. Let $\mathcal{L}$ be the set of semantic categories. Each category in $\mathcal{L}$ is a label for a certain concept, such as "person's name" or "locations". We define the category-based policy Ψ_C for categories $\mathcal{L}$ as

$$\Psi_C = \{w \in \Psi_F \mid l(w) \in \mathcal{L}\},$$

where $l : \mathbb{W} \to \mathcal{L} \cup \{\varepsilon\}$ is a labeling function that assigns a category from the set $\mathcal{L}$ to a token $w \in \mathbb{W}$. In the case that a token does not match any category, it is mapped to ε. We assume that the labeling function is implemented with semantic text classification [36]. Here, we use Qwen2.5-7B to classify the tokens into categories based on their semantics.

B Privacy Metrics Definitions

B.1 Re-identification score (RI)

This metric is calculated by simulating an attack on the masked question. More specifically, an LLM is prompted with the masked NL question and database schema and instructed to infer the masked tokens. Then, the percentage of the masked tokens that could not be re-identified is calculated as the score. Formally, let $\mathbb{W}$ be the set of all natural language tokens and $\mathbb{S} \subseteq \mathbb{W}$ be the set of masked symbols. Let $\mathcal{Q} = w_1, w_2, \ldots, w_n$ be an NL question where $w_i \in \mathbb{W}$ for $1 \leq i \leq n$. We define $\mathcal{Q}_s = \{w_i \mid 1 \leq i \leq n\}$ as the set of tokens used in $\mathcal{Q}$. We assume that questions do not include masked tokens, i.e., $\mathcal{Q}_s \cap \mathbb{S} = \emptyset$. For a database schema $\mathcal{S}$, we define masking functions as $f : \mathbb{W} \to \mathbb{W}$, which maps each token of a question to either a masked symbol or the token itself. Let f^{-1} denote the inverse of f, which returns the original token for a masked token. For a question $\mathcal{Q}$, let $f(\mathcal{Q}) = f(w_1), f(w_2), \ldots, f(w_n)$ be its masked version. For a question $\mathcal{Q}$ and masking function f, let $\mathcal{Q}' = f(\mathcal{Q})$ be its masked version, and G be the re-identified question generated by a simulated attack on $\mathcal{Q}'$. Let $\mathcal{Q}'_s$ and G_s be the set of tokens for $\mathcal{Q}'$ and G, respectively. We define the re-identification score as follows:

$$R_f(\mathcal{Q}'_s, G_s) = 1 - \frac{|\{w \in \mathcal{Q}_s \mid f(w) \in \mathbb{S}\} \cap G_s|}{|\mathcal{Q}'_s \cap \mathbb{S}|} \tag{1}$$

where the set builder notation is used to denote the set of original tokens in the $\mathcal{Q}_s$ that have been masked by f, and $\mathcal{Q}'_s \cap \mathbb{S}$ is the set of all masked tokens in $\mathcal{Q}'_s$. A score of 1 means none of the masked tokens could be inferred by the attacker. Here, we use exact string matching and not semantic or distance-based methods.

We calculate the re-identification score for each masked question and report the average over all questions as the score for the whole dataset.

B.2 Masking Recall (MR)

Masking recall is defined as the ratio of the masked tokens to the total number of tokens in the ground-truth masking. Formally, let f, f_g be two masking functions where f_g is the ground-truth, i.e.,

[56] 江狗战, 张文辉, 张正, 薛焕然, 张瑶, 吴烨. Portcullis: 一个可扩展且可验证的第三方大语言模型推理隐私网关. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(1):1022–1030, 2025. doi: 10.1609/aaai.v39i1.32088. [57] 张斌, 叶宇潇, 杜国庆, 胡晓如, 李志帅, 杨孙, 刘奇·哈罗德, 赵瑞, 李子悦, 毛航宇. 大语言模型文本到SQL能力的基准测试: 一项全面评估. *arXiv:2403.02951*, 2024. [58] 维克多·钟, 熊采明, 理查德·索彻. Seq2sql: 使用强化学习从自然语言生成结构化查询. *arXiv:1709.00103*, 2017. [59] 朱晓虎, 李倩, 崔立贞, 刘永康. 大语言模型增强的文本到SQL生成: 一项综述. *arXiv:2410.06011*, 2024.

基于类别的策略定义

令 Ψ_F 为完整策略，其中整个模式和数据库值被抽象化。令 $\mathcal{L}$ 为语义类别集合。$\mathcal{L}$ 中的每个类别都是某个概念的标签，例如“人名”或“地点”。我们将类别 $\mathcal{L}$ 的基于类别的策略 Ψ_C 定义为

$$\Psi_C = \{w \in \Psi_F \mid l(w) \in \mathcal{L}\},$$

其中 $l : \mathbb{W} \to \mathcal{L} \cup \{\varepsilon\}$ 是一个标注函数，用于将标记 $w \in \mathbb{W}$ 分配到集合 $\mathcal{L}$ 中的某个类别。如果标记不匹配任何类别，则将其映射到 ε。我们假设该标注函数是通过语义文本分类 [36] 实现的。在这里，我们使用 Qwen2.5-7B 根据标记的语义将其分类到各个类别中。

B 隐私指标定义

B.1 重新识别得分（RI）

该指标通过模拟对被掩码的问题进行攻击来计算。更具体地说，LLM 会被提示输入被掩码的自然语言问题和数据库模式，并被指示推断被掩码的标记。然后，将无法重新识别的被掩码标记的百分比计算为分数。形式上，设 $\mathbb{W}$ 为所有自然语言标记的集合，$\mathbb{S} \subseteq \mathbb{W}$ 为被掩码符号的集合。设 $\mathcal{Q} = w_1, w_2, \ldots, w_n$ 为一个自然语言问题，其中 $w_i \in \mathbb{W}$ 对于 $1 \leq i \leq n$。我们定义 $\mathcal{Q}_s = \{w_i \mid 1 \leq i \leq n\}$ 为在 $\mathcal{Q}$ 中使用的标记集合。我们假设问题中不包含被掩码的标记，即 $\mathcal{Q}_s \cap \mathbb{S} = \emptyset$。对于数据库模式 $\mathcal{S}$，我们定义掩码函数为 $f\colon\ \mathbb{W} \to \mathbb{W}$，它将问题的每个标记映射为被掩码符号或标记本身。设 f^{-1} 表示 f 的逆函数，它返回被掩码标记的原始标记。对于问题 $\mathcal{Q}$，设 $f(\mathcal{Q}) = f(w_1), f(w_2), \ldots, f(w_n)$ 为其被掩码版本。对于问题 $\mathcal{Q}$ 和掩码函数 f，设 $\mathcal{Q}' = f(\mathcal{Q})$ 为其被掩码版本，G 为重新识别的版本。

$$R_f(\mathcal{Q}'_s, G_s) = 1 - \frac{|\{w \in \mathcal{Q}_s \mid f(w) \in \mathbb{S}\} \cap G_s|}{|\mathcal{Q}'_s \cap \mathbb{S}|} \tag{1}$$

其中使用集合构造符号表示在 $\mathcal{Q}_s$ 中被 f 掩盖的原始标记集合，且 $\mathcal{Q}'_s \cap \mathbb{S}$ 是 $\mathcal{Q}'_s$ 中所有被掩盖标记的集合。得分为 1 表示攻击者无法推断出任何被掩盖的标记。在这里，我们使用精确字符串匹配，而不是语义或基于距离的方法。

我们计算每个被屏蔽问题的重新识别分数，并将所有问题的平均值作为整个数据集的分数。

B.2 掩蔽回忆 (MR)

掩码召回率被定义为被掩码的标记数量与真实掩码中标记总数的比率。形式上，设 f, f_g 为两个掩码函数，其中 f_g 为真实掩码，即，

a mapping that masks all ground-truth sensitive tokens. For a question $\mathcal{Q}$ and its set of tokens $\mathcal{Q}_s$, we define the masking recall as follows:

$$M_f(\mathcal{Q}_s) = \frac{\{w \in \mathcal{Q}_s \mid f_g(w) \in \mathbb{S} \wedge f(w) \in \mathbb{S}\}}{|f_g(\mathcal{Q}_s)|} \tag{2}$$

A score of 1 indicates that the tool successfully masked all required tokens. Similarly, we calculate the average of the scores to report it for a dataset.

C Ablation Study

To quantify the contribution of each component in the MaskSQL pipeline, we conducted an ablation study by removing each stage and measuring how accuracy and privacy scores are changed. The results, presented in Table 2, demonstrate that each component plays a crucial role in the MaskSQL pipeline (Figure 1).

Stage	EX	RI	MR
Complete Pipeline	**55.66%**	75.47%	61.36%
Schema Filtering	53.66%	76.49%	58.56%
Value Detection	53.33%	82.17%	64.45%
Value Linking	52.33%	74.97%	48.64%
Abstraction	47.00%	**86.79%**	**65.45%**
LLM Correction	51.33%	75.47%	61.36%
SQL Reconstruction	34.33%	75.47%	61.36%
SLM Correction	35.33%	75.47%	61.36%

Table 2: Ablation study results showing the effect of removing each component of the MaskSQL pipeline on the final execution accuracy (EX), re-identification score (RI), and masking recall (MR).

Schema Filtering. For this experiment, we remove the schema filtering at the beginning of the pipeline. As a result, the whole database schema is always passed to the further stages. As shown in the table, removing schema linking step decreases the execution accuracy and masking recall.

Value Detection. Here, we remove the separate stage for detecting the values in the question. Removing this stage slightly decreases the execution accuracy.

Value Linking. In this experiment, we remove the separate stage for linking the values of the question. While value linking slightly decreases the execution accuracy, it has a significant negative impact on the masking recall and decreases it by $\sim 13\%$.

Abstraction. In the MaskSQL pipeline, we use separate stages for schema linking and abstraction. To measure the effect of this separation, we removed the schema linking stage and used an SLM to abstract away the question with respect to the database schema. While abstraction with SLM and without a separate schema linking step improves the privacy measures, it decreases the execution accuracy by $\sim 8\%$.

LLM Correction. By removing the abstract SQL correction stage, we see a slight decrease in execution accuracy.

SQL Reconstruction. Similar to abstraction, to measure the effect of the deterministic unmasking process, we prompt SLM to unmask the SQL query and also correct the errors in the query if needed. As shown in the table, replacing the deterministic unmasking stage with language model prompting significantly reduces the execution accuracy by $\sim 21\%$.

SLM correction. The final correction stage prompts the SLM to fix errors in the reconstructed SQL query, and, as shown in the table, it has a significant impact on the execution accuracy. Removing this correction step causes a $\sim 20\%$ drop in execution accuracy.

一个映射，它会屏蔽所有真实的敏感词令。对于一个问题 $\mathcal{Q}$ 及其词令集合 $\mathcal{Q}_s$，我们将屏蔽召回定义如下：

$$M_f(\mathcal{Q}_s) = \frac{\{w \in \mathcal{Q}_s \mid f_g(w) \in \mathbb{S} \wedge f(w) \in \mathbb{S}\}}{|f_g(\mathcal{Q}_s)|} \tag{2}$$

分数为1表示该工具成功地屏蔽了所有需要的标记。同样地，我们计算分数的平均值以报告数据集的情况。

C 消融研究

为了量化 MaskSQL 流水线中每个组件的贡献，我们进行了消融研究，通过移除每个阶段并测量准确性和隐私分数的变化。结果如表 2 所示，表明每个组件在 MaskSQL 流水线（图 1）中都起着关键作用。

Stage	EX	RI	MR
Complete Pipeline	**55.66%**	75.47%	61.36%
Schema Filtering	53.66%	76.49%	58.56%
Value Detection	53.33%	82.17%	64.45%
Value Linking	52.33%	74.97%	48.64%
Abstraction	47.00%	**86.79%**	**65.45%**
LLM Correction	51.33%	75.47%	61.36%
SQL Reconstruction	34.33%	75.47%	61.36%
SLM Correction	35.33%	75.47%	61.36%

表 2：消融研究结果显示移除 MaskSQL 流水线中各个组件对最终执行准确率（EX）、重新识别评分（RI）和掩码召回率（MR）的影响。

模式过滤。在本实验中，我们在管道的开始阶段移除了模式过滤。因此，整个数据库模式始终会传递到后续阶段。如表中所示，移除模式链接步骤会降低执行准确率和掩码召回率。

值检测。在这里，我们去掉了用于检测问题中数值的单独阶段。去掉这个阶段会略微降低执行准确性。

值链接。在这个实验中，我们移除了用于链接问题值的单独阶段。虽然值链接会略微降低执行准确性，但它对掩码召回率有显著的负面影响，降低了 $\sim 13\%$ 。

抽象。在 MaskSQL 流程中，我们为模式链接和抽象使用了不同的阶段。为了衡量这种分离的效果，我们移除了模式链接阶段，并使用 SLM 根据数据库模式抽象化问题。虽然使用 SLM 抽象化并且没有单独的模式链接步骤可以提高隐私措施，但它会使执行准确率降低约 8%（$\sim$）。

大型语言模型纠正。通过移除抽象 SQL 纠正阶段，我们看到执行准确率略有下降。

SQL 重建。类似于抽象，为了衡量确定性去掩码过程的效果，我们提示 SLM 去去掩码 SQL 查询，并在需要时纠正查询中的错误。如表中所示，用语言模型提示替代确定性去掩码阶段会显著降低执行准确率 $\sim 21\%$。

SLM 校正。最终校正阶段提示 SLM 修正重建的 SQL 查询中的错误，如表中所示，它对执行准确性有显著影响。移除此校正步骤会导致执行准确性下降 $\sim 20\%$。

D Discussion of Experimental Results

Table 1 shows the execution accuracy of different frameworks. As shown in the figure, direct prompting to GPT-4.1 outperforms all evaluated models. This is a surprising result, as both DIN-SQL and DAIL-SQL are also utilizing GPT-4.1 as the backend LLM and performing additional processing and improvements to enhance accuracy. The direct method uses the prompt shown in Figure F.1, but with the concrete examples instead of the abstract ones shown in the figure. Note that the BIRD benchmark website reports DIN-SQLand DAIL-SQLsubmissions based on GPT-4, rather than the later version GPT-4.1. Moreover, our experiments use a different evaluation set. These differences explain why GPT-4.1 surpassed them in our results, even though they achieved top ranks on the official BIRD leaderboard.

E Additional Figures

```
-- NL Question:
-- List the power of superheroes with height greater than 80% of the average height
↪  of all superheroes.
-- Power of superheroes refers to power_name;
-- Height greater than 80% of the average height of all superheroes = height_cm >
↪  MULTIPLY(AVG(height_cm), 0.8);


-- Qwen2.5 Output (Incorrect):
SELECT
    [superpower].[power_name]
FROM
    [hero_power]
JOIN
    [superpower] ON [hero_power].[power_id] = [superpower].[id]
JOIN
    [superhero] ON [hero_power].[hero_id] = [superhero].[id]
GROUP BY
    [superpower].[power_name]
HAVING
    [superhero].[height_cm] > AVG([superhero].[height_cm]) * 0.8;

-- GPT-4.1 Output (Correct):
SELECT
    [superpower].[power_name]
FROM
    [hero_power]
JOIN
    [superhero] ON [hero_power].[hero_id] = [superhero].[id]
JOIN
    [superpower] ON [hero_power].[power_id] = [superpower].[id]
WHERE
    [superhero].[height_cm] > (
        0.8 * (SELECT AVG([superhero].[height_cm]) FROM [superhero])
    );
```

Figure 3: An example of an SQL query that requires advanced constructs like nested queries, which Qwen2.5-7B failed to handle properly. This example is extracted from our experiments.

F Prompt Templates

F.1 SQL Generation Prompt

```
You are an SQL generation assistant. Given

(1) NL Question: a natural-language question about a dataset and
```

D 实验结果讨论

表 1 显示了不同框架的执行准确率。如图所示，直接向 GPT-4.1 提示的效果优于所有评估模型。这是一个令人惊讶的结果，因为 DIN-SQL 和 DAIL-SQL 同样使用 GPT-4.1 作为后端大语言模型，并进行了额外处理和改进以提高准确性。直接方法使用图 F.1 中展示的提示，但使用具体示例而非图中显示的抽象示例。请注意，BIRD 基准网站报告的 DIN-SQL 和 DAIL-SQL 提交是基于 GPT-4，而非后续版本 GPT-4.1。此外，我们的实验使用了不同的评估集。这些差异解释了尽管它们在官方 BIRD 排行榜上取得了最高排名，但在我们的结果中 GPT-4.1 超过了它们。

附加图表

-- NL 问题：-- 列出身高大于所有超级英雄平均身高 80% 的超级英雄的能力。↪ -- 超级英雄的能力指的是 power_name；-- 身高大于所有超级英雄平均身高的 80% = height_cm > MULTIPLY(AVG(height_cm), 0.8); ↪

-- Qwen2.5 输出（不正确）： SELECT [superpower].[power_name] FROM[hero_power] JOIN[superpower] ON [hero_power].[power_id] = [superpower].[id] JOIN[superhero] ON [hero_power].[hero_id] = [superhero].[id] GROUP BY [superpower].[power_name] HAVING [superhero].[height_cm] > AVG([superhero].[height_cm]) * 0.8;

-- GPT-4.1 输出（正确）： SELECT [superpower].[power_name] FROM[hero_power] JOIN[superhero] ON [hero_power].[hero_id] = [superhero].[id] JOIN[superpower] ON [hero_power].[power_id] = [superpower].[id] WHERE[superhero].[height_cm] >（ 0.8 * (SELECT AVG([superhero].[height_cm]）FROM [superhero]));

图3：一个需要高级结构（如嵌套查询）的SQL查询示例，Qwen2.5-7B未能正确处理。此示例摘自我们的实验。

F 提示模板

F.1 SQL 生成提示

你是一个SQL生成助手。给定

(1) NL 问题：关于数据集的自然语言问题以及

NL Question ($\mathcal{Q}$)

```
How many patients did the New York
↪  Hospital admit with HIV status as
↪  positive?
```

Abstract NL Question ($\mathcal{Q}'$)

```
How many T1 did the V1 T3 with C3 as
↪  V2?; V1 is a value of the column C7;
↪  V2 is a value of the column C3
```

SQL Query ($\mathcal{Y}$)

```
SELECT count(P.pid)
FROM Patient AS P
JOIN Admission AS A ON P.pid = A.pid
JOIN Hospital AS H ON A.hid = H.hid
WHERE H.name = "New York Hospital"
AND P.hiv_status = 1
```

Abstract SQL Query ($\mathcal{Y}'$)

```
SELECT count(T1.C1)
FROM T1
JOIN T3 ON T1.C1 = T3.C10
JOIN T2 ON T3.C11 = T2.C6
WHERE  T2.C7 = 'V1'
AND T1.C3 = 'V2';
```

Database Schema ($\mathcal{S}$)

```
'Patients':
    'pid':
        primary_key: true
        type: integer
    'name': text
    'hiv_status': integer
    'diagnosis': text
    'treatment': text
'Hospital':
    'hid':
        primary_key: true
        type: integer
    'name': text
    'address': text
'Admissions':
    'aid':
        primary_key: true
        type: integer
    'pid':
        foreign_key: 'Patients.pid'
        type: integer
    'hid':
        foreign_key: 'Hospital.hid'
        type: integer
    'date': date
```

Abstract Database Schema ($\mathcal{S}'$)

```
'T1':
    'C1':
        primary_key: true
        type: integer
    'C2': text
    'C3': integer
    'C4': text
    'C5': text
'T2':
    'C6':
        primary_key: true
        type: integer
    'C7': text
    'C8': text
'T3':
    'C9':
        primary_key: true
        type: integer
    'C10':
        foreign_key: 'T1.C1'
        type: integer
    'C11':
        foreign_key: 'T2.C6'
        type: integer
    'C12': date
```

Figure 4: Abstract question, database schema, SQL query for Example 1.

```
(2) DB Schema: the database's schema expressed in YAML

produce a single SQL SELECT statement that answers the question.

Input Format:
- DB Schema: given in YAML format, where top-level keys are table names;
↪  each table lists its columns and their data types.
- Column names are case-sensitive exactly as shown in the schema.
- Each column might be a primary key or a foreign key.
- For foreign key columns, the fully qualified name of the referenced
↪  column is given.

Output Rules:
```

NL 问题（$\mathcal{Q}$）

纽约医院收治了多少名HIV呈阳性的患者?
↪
↪

抽象自然语言问题 ($\mathcal{Q}'$)

V1 使用 C3 作为 V2 对 T1 做了多少 T3? V1 是列 C7 的一个值；V2 是列 C3 的一个值 ↪
↪

SQL 查询（$\mathcal{Y}$）

```
SELECT count(P.pid) FROM Patient AS P JO
IN Admission AS A ON P.pid = A.pid JOIN H
ospital AS H ON A.hid = H.hid WHERE H.na
me = "纽约医院" AND P.hiv_status = 1
```

抽象 SQL 查询 ($\mathcal{Y}'$)

```
SELECT count(T1.C1) FROM
T1 JOIN T3 ON T1.C1 = T3.C1
0 JOIN T2 ON T3.C11 = T2.C6
WHERE T2.C7 = 'V1' AND T1.
C3 = 'V2';
```

数据库架构（$\mathcal{S}$）

'患者': 'pid': 主键: true 类型: 整数 '姓名': 文本 'HIV状态': 整数 '诊断': 文本 '治疗': 文本 '医院': 'hid': 主键: true 类型: 整数 '名称': 文本 '地址': 文本 '入院记录': 'aid': 主键: true 类型: 整数 'pid': 外键: '患者.pid' 类型: 整数 'hid': 外键: '医院.hid' 类型: 整数 '日期': 日期

抽象数据库模式（$\mathcal{S}'$） 'T1':'C1':主键: true 类型: 整数 'C2': 文本 'C3': 整数 'C4': 文本 'C5': 文本 'T2':'C6':主键: true 类型: 整数 'C7': 文本 'C8': 文本 'T3':'C9':主键: true 类型: 整数 'C10': 外键: 'T1.C1' 类型: 整数 'C11': 外键: 'T2.C6' 类型: 整数 'C12': 日期

无需图片：抽象问题、数据库模式、Exa 的 SQL 查询 示例 1。

(2) 数据库模式：用 YAML 表示的数据库模式

生成一个能够回答该问题的单个 SQL SELECT 语句。

输入格式:
- 数据库架构：以 YAML 格式给出，顶级键为表名；每个表列出其列及其数据类型。↪
- 列名区分大小写，必须与架构中显示的一致。- 每个列可能是主键或外键。- 对于外键列，给出引用列的完全限定名称。↪

输出规则:

```
- Table and column names specified in the database schema are already
↪  wrapped in brackets. You should use them with the brackets.
You should not remove the brackets when using them in the SQL.
- Each reference to a table or column name should be of the form
↪  [table_name] or [table_name].[column_name].
- Output ONLY the SQL query (no extra explanation or text).
- Use fully qualified column names: table.column.
- Only reference tables/columns that exist in the provided schema.
- Do not include any comments.
- For column names with spaces, wrap them in backticks, e.g., "WHERE `car
↪  model` = 'bar'" instead of "WHERE car model = 'bar'".

Here are some examples:

...
-----------------------------------
#### Example 2

**NL Question:**
Among the [V1] [T1], how many of them have a [C2] of [V2]? [V1] refers to
↪  [C2] = [V1].

**DB Schema:**
[T1]:
    [C1]: text
    [C2]: real
    [C3]:
        primary_key: true
        type: integer

**Output:**
`SELECT COUNT(*) FROM [T1] WHERE [T1].[C1] = [V1] AND [C2] = [V2]

-----------------------------------
...

Now, generate an SQL query for the following question and database schema:
Inputs:
NL Question: {NL_QUESTION}
DB Schema: {DB_SCHEMA}
```

F.2 Self-Correction Prompt (Abstract)

```
You are an SQL database expert tasked with debugging an SQL query.
A previous attempt to predict an SQL query given a masked NL question and
↪  DB schema did not yield the correct results in some cases.
Either due to errors in execution or because the result returned was empty
↪  or unexpected.
Your task is to analyze the masked SQL query given the corresponding
↪  database schema and the NL question.
and fix any errors in the query if they exist.
You should then provide a corrected version of the SQL query.
Note that there may be errors in how the NL question tokens were linked and
↪  masked with the database schema elements.
As a result, the masked SQL query might contain inaccuracies based on these
↪  incorrect mappings,
and part of your task is to consider these issues as well.

**Procedure:**
1. Review Database Schema:
```

- 数据库模式中指定的表和列名已经用方括号括起来。你应该带着方括号使用它们。↪在 SQL 中使用它们时不应去掉方括号。- 每次引用表或列名应采用 [table_name] 或 [table_name].[column_name] 的形式。↪- 仅输出 SQL 查询（不包含额外说明或文本）。- 使用完整限定的列名：table.column。- 仅引用提供的模式中存在的表/列。- 不要包含任何注释。

- 对于带空格的列名，请将它们用反引号括起来，例如，“WHERE `car model` =` 'bar'”而不是“WHERE car model = 'bar'”。↪

这里有一些例子：

... ----------

示例 2

自然语言问题：
在这些 [V1] [T1] 中，有多少个拥有 [V2] 的 [C2]？[V1] 指的是 [C2] = [V1]。↪

数据库模式: [T1]:[C1]: 文本 [C2]: 实数 [C3]:主键: true
类型: 整数

输出:
`SELECT COUNT(*) FROM [T1] WHERE [T1].[C1] = [V1] AND [C2] = [V2]

---------- ...

现在，为以下问题和数据库模式生成一个 SQL 查询：输入：自然语言问题：{NL_QUESTION} 数据库模式：{DB_SCHEMA}

F.2 自我纠正提示（摘要）

您是一名SQL数据库专家，负责调试SQL查询。
之前尝试根据带有掩码的自然语言问题和数据库模式预测 SQL 查询，在某些情况下未能得到正确的结果。↪
要么是由于执行错误，要么是因为返回的结果为空或出乎意料。↪

你的任务是根据相应的数据库模式和自然语言问题分析给定的屏蔽 SQL 查询。↪

并修复查询中存在的任何错误。
然后您应该提供经过纠正的 SQL 查询版本。
请注意，自然语言问题的标记与数据库模式元素的关联和掩码可能存在错误。↪

因此，基于这些错误映射，被屏蔽的 SQL 查询可能包含不准确之处，↪

你的部分任务也是考虑这些问题。

程序：
1. 审查数据库模式：

```
    - Examine the database schema to understand the database structure.
    - Database schema is given in YAML format, where top-level keys are
    ↪  table names; each table lists its columns and their data types.
    - Each column might be a primary key or a foreign key.
2. Analyze Query Requirements:
    - NL Question: Consider what information the query is supposed to
    ↪  retrieve.
    - Predicted SQL Query: Review the SQL query that was previously
    ↪  predicted and might have led to an error or incorrect result.
3. Correct the Query:
    - Modify the SQL query to address the identified issues, ensuring it
    ↪  correctly fetches the requested data according to the database
    ↪  schema and query requirements.

**Output Format:**
Present your corrected query as a single line of SQL code.
Ensure there are no line breaks within the query.
Do not include any explanations, comments, or extra text.

Here are some examples:

-------------------------------
Example 1:

NL Question:
Among the [V1] [T1], how many of them have [C2] of zero? [V1] is a
↪  nationality of [C1] = [V1];

Database Schema:
'[T1]':
    '[C1]': text
    '[C2]': real
    '[C3]':
        primary_key: true
        type: integer

The predicted SQL query was:
SELECT COUNT(*) FROM [T1] WHERE [T1].[C1] = '[V1]' AND [C2] = 0

The corrected SQL query is:
SELECT COUNT(*) FROM [T1] WHERE [T1].[C1] = '[V1]' AND [C2] = 0
-------------------------------

======= Your task =======
**************************
Database schema:
{schema}
**************************
The original question is:
NL Question: {question}
The predicted SQL query: {sql}
**************************
Based on the NL question, database schema, and the previously predicted SQL
↪  query,
Analyze the query and question, and fix the SQL query if needed.
```

F.3 Self-Correction Prompt (Concrete)

```
You are an SQL database expert tasked with correcting an SQL query.
```

- 检查数据库模式以了解数据库结构。- 数据库模式以 YAML 格式提供，顶层键是表名；每个表列出其列及其数据类型。↪- 每个列可能是主键或外键。

2. 分析查询需求：- 自然语言问题：考虑查询应该检索哪些信息。↪- 预测的SQL查询：查看之前预测的SQL查询，它可能导致错误或不正确的结果。↪

三修正查询：- 修改 SQL 查询以解决已识别的问题，确保它能够根据数据库模式和查询要求正确获取所请求的数据。↪
↪

输出格式：
将您纠正后的查询以单行 SQL 代码的形式呈现。
确保查询中没有换行符。做
不包括任何解释、评论或额外内容 一段文字。

这里有一些例子：

示例 1：

自然语言问题：
在[V1][T1]中，有多少人的[C2]为零？[V1]是[C1]=[V1]的国籍；↪

数据库模式：'[T1]'：'[C1]'：
文本 '[C2]'：实数 '[C3]'：主键
：真 类型：整数

预测的 SQL 查询是：
SELECT COUNT(*) FROM [T1] WHERE [T1].[C1] = '[V1]' AND [C2] = 0

修正后的 SQL 查询是：
SELECT COUNT(*) FROM [T1] WHERE [T1].[C1] = '[V1]' AND [C2] = 0
---------- ======= 你
的任务 =======

****
数据库模式：
{schema}
****
原始问题是：
NL 问题：{question}
预测的 SQL 查询：{sql}
****
基于自然语言问题、数据库模式以及先前预测的 SQL 查询，↪

分析查询和问题，并在需要时修复SQL查询。

F.3 自我纠正提示（具体）

您是一名SQL数据库专家，负责纠正SQL查询。

```
A previous attempt to run a query did not yield the correct results,
either due to errors in execution or because the result returned was empty
↪  or unexpected.
Your role is to analyze the error based on the provided database schema and
↪  the details
of the failed execution, and then provide a corrected version of the SQL
↪  query.

**Procedure:**
1. Review Database Schema:
    - Examine the database schema to understand the database structure.
    - Iterate through each column and table name in the schema to make sure
    ↪  that it is correct.
    - Some table or column names may have white space; you should not
    ↪  change these and use a different name.
    - Database schema is given in YAML format, where top-level keys are
    ↪  table names; each table lists its columns and their data types.
    - Column names are case-sensitive, exactly as shown in the schema.
    - Each column might be a primary key or a foreign key.
    - For foreign key columns, the fully qualified name of the referenced
    ↪  column is given.
2. Analyze Query Requirements:
    - Original Question: Consider what information the query is supposed to
    ↪  retrieve.
    - Executed SQL Query: Review the SQL query that was previously executed
    ↪  and led to an error or incorrect result.
    - Execution Result: Analyze the outcome of the executed query to
    ↪  identify why it failed (e.g., syntax errors, incorrect column
    ↪  references, logical mistakes).
3. Correct the Query:
    - Modify the SQL query to address the identified issues, ensuring it
    ↪  correctly fetches the requested data according to the database
    ↪  schema and query requirements.
    - For column names with spaces, wrap them in backticks, e.g., "WHERE
    ↪  `car model` = 'bar'" instead of "WHERE car model = 'bar'".

**Output Format:**
Present your corrected query as a single line of SQL code.
Ensure there are no line breaks within the query.
Do not include any explanations, comments, or extra text.

Here are some examples:

--------------------------------
Example 1:
Question:
Among the German customers, how many of them have a credit limit of zero?
↪  German is a nationality of the country = 'Germany'; CREDITLIMIT = 0.

Database Schema:
'[customers]':
    '[country]': text
    '[creditlimit]': real
    '[customernumber]':
        primary_key: true
        type: integer

The SQL query executed was:
SELECT COUNT(*) FROM [customers] WHERE [customers].[country] = 'German' AND
↪  [creditlimit] = 0
```

之前尝试运行查询未能得到正确的结果，可能是由于执行错误，或者返回的结果为空或不符合预期。↪你的角色是根据提供的数据库模式和失败执行的详细信息分析错误，↪然后提供修正后的 SQL 查询版本。↪

程序：
1. 审查数据库模式：
    - 检查数据库模式以了解数据库结构。 - 遍历模式中的每一列和表名，以确保它们是正确的。 ↪- 有些表或列名可能包含空格；你不应该更改这些名称，也不应使用不同的名称。 ↪- 数据库模式以 YAML 格式给出，顶级键是表名；每个表列出其列和数据类型。 ↪- 列名区分大小写，完全按模式中显示的名称。 - 每一列可能是主键或外键。 - 对于外键列，给出被引用列的完全限定名称。 ↪

2. 分析查询需求：- 原始问题：考虑查询应该检索哪些信息。↪- 已执行的 SQL 查询：查看之前执行的 SQL 查询，该查询导致错误或结果不正确。↪- 执行结果：分析已执行查询的结果，以确定失败的原因（例如，语法错误、列引用不正确、逻辑错误）。↪
    ↪

3. 修改查询：
    - 修改 SQL 查询以解决已识别的问题，确保其根据数据库模式和查询要求正确获取请求的数据。↪- 对于带有空格的列名，用反引号将其括起来，例如，"WHERE ĉar model` = 'baŕ"" 而不是 "WHERE car model = 'bar'"。↪

输出格式：
将您纠正后的查询以单行 SQL 代码的形式呈现。
恩斯确保查询中没有换行符。
不要包含任何解释、评论或额外文字。

这里有一些例子：

示例 1：
问题：
在德国客户中，有多少人的信用额度为零？德国是国家 = 'Germany' 的国籍；信用额度 = 0。↪

数据库模式：'[customers]': '[co
untry]': 文本 '[creditlimit]': 实数
'[customernumber]': 主键: 是 类
型: 整数

执行的 SQL 查询是：
SELECT COUNT(*) FROM [customers] WHERE [customers].[country] = '德国' AND
    [信用额度] = 0 ↪

```
Output:
SELECT COUNT(*) FROM [customers] WHERE [customers].[country] = 'Germany'
↪  AND [creditlimit] = 0

The execution result:
[]
--------------------------------

======= Your task =======
**************************
Database schema:
{schema}
**************************
The original question is:
Question: {question}
The SQL query executed was: {sql}
The execution result: {exec_res}
**************************
Based on the question, table schema, and the previous query, analyze the
↪  result and try to fix the query.
```

输出:
SELECT COUNT(*) FROM [customers] WHERE [customers].[country] = '德国' AND [credi
tlimit] = 0 ↪

执行结果: []----------

======= 你的任务 ======= ********** 数据库模式: {schema} ********** 原始问题是: 问题: {question} 执行的 SQL 查询是: {sql} 执行结果: {exec_res} ********** 根据问题、表模式和之前的查询，分析结果并尝试修正查询。 ↪